

DEPARTMENT OF
INFORMATION TECHNOLOGY AND ELECTRICAL ENGINEERING

Fall Semester 2025

CachePool: A scalable domain-specific cache-coherent manycore architecture with scalar-vector capability for dataplane

Master Thesis



Ho Tin Hung

hohung@student.ethz.ch

23.03.2026

Advisors: Zexin Fu, zexifu@iis.ee.ethz.ch
Diyoun Shen, dishen@iis.ee.ethz.ch

Professor: Prof. Dr. Luca Benini, lbenini@iis.ee.ethz.ch

Abstract

CachePool is a scalable manycore architecture with scalar–vector capability targeting irregular workloads such as Radio Link Control (RLC) dataplanes. While the baseline design provides a shared on-tile cache structure, such an organization limits the ability to exploit locality close to each core complex and becomes increasingly challenging as the architecture scales. This work is motivated by the need for a memory hierarchy that better supports sparse, synchronization-heavy, and latency-sensitive access patterns.

The objective of this thesis is to extend CachePool with a private-cache hierarchy and a lightweight hardware-managed coherence mechanism. A private L1 data cache for each core complex (CC) is introduced, backed by a logically shared, inclusive, multi-banked L2 cache, and a directory-based coherence protocol is developed. The main challenge lies in integrating the new hierarchy into the existing CachePool architecture: the private L1 must be interfaced with the scalar–vector core complex and the tile interconnect, the inclusive L2 must be extended with directory information, and the coherence protocol must remain lightweight and scalable enough for future multi-tile systems.

This work adopts HPDcache as the per-CC private L1 cache and reuses the existing shared data cache as the inclusive L2. A directory controller is embedded at the L2-bank level by extending the cache metadata with coherence state and sharer information. The resulting design is verified using coherence-oriented functional test cases and evaluated using representative software kernels, including floating-point compute workloads and RLC dataplane applications.

The results shows a performance degradation caused by L1 miss penalties and coherence management. From a back-end perspective, the implementation incurs an area cost of 2638kGE, of which 2200kGE is attributed to the private L1 caches and only 154kGE to the added coherence support. This demonstrates the feasibility of extending CachePool toward a coherent private-cache hierarchy and provides a practical foundation for future exploration of more advanced write policies and larger multi-tile coherent systems.

Acknowledgments

First and foremost, I would like to express my deepest gratitude to Professor Dr. Luca Benini for providing me with the opportunity to conduct my research. Your vision, leadership, and the high standards of excellence maintained have been fundamental to my academic growth.

I am especially grateful to my advisors, Zexin Fu and Diyou Shen. This thesis would not have been possible without your guidance, patience, and technical insights. Thank you for stepping in whenever I hit a wall, and for the efforts of brainstorming. Your support went far beyond mere supervision, and I am lucky to have had you in my corner.

I would also like to thank Frank K. Gürkaynak and the Design Zentrum (DZ) for their indispensable technical support. Access to the specialized resources and infrastructure managed by the DZ team was critical to the realization of this project; thank you for ensuring the wheels kept turning behind the scenes.

I would like to extend my sincere thanks to the OpenHW Group for their work that served as the foundation for this work. In particular, I am deeply grateful to the author of HPDcache, César Fuguet, for the exceptional work and for the advice provided.

To the IIS research group and the contributors of the PULP platform, thank you for the incredible work and resources upon which this project is built. Being part of it has been a privilege, and I am grateful for the collaborative environment and the shared expertise.

Finally, I want to thank my family and friends for their unwavering encouragement and for keeping me grounded throughout this process.



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Declaration of originality

The signed declaration of originality is a component of every written paper or thesis authored during the course of studies. **In consultation with the supervisor**, one of the following two options must be selected:

- ☐ I hereby declare that I authored the work in question independently, i.e. that no one helped me to author it. Suggestions from the supervisor regarding language and content are excepted. I used no generative artificial intelligence technologies¹.
- ☒ I hereby declare that I authored the work in question independently. In doing so I only used the authorised aids, which included suggestions from the supervisor regarding language and content and generative artificial intelligence technologies. The use of the latter and the respective source declarations proceeded in consultation with the supervisor.

Title of paper or thesis:

CachePool: A scalable domain-specific cache-coherent manycore architecture with scalar-vector capability for dataplane

Authored by:

If the work was compiled in a group, the names of all authors are required.

Last name(s):

Hung

First name(s):

Ho Tin

With my signature I confirm the following:

- I have adhered to the rules set out in the [Citation Guidelines](#).
- I have documented all methods, data and processes truthfully and fully.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for originality.

Place, date

Zürich, 23.03.2026

Signature(s)

If the work was compiled in a group, the names of all authors are required. Through their signatures they vouch jointly for the entire content of the written work.

¹ For further information please consult the ETH Zurich websites, e.g. <https://ethz.ch/en/the-eth-zurich/education/ai-in-education.html> and <https://library.ethz.ch/en/researching-and-publishing/scientific-writing-at-eth-zurich.html> (subject to change).

Contents

1. Introduction	1
2. Background	2
2.1. Radio Access Network (RAN)	2
2.1.1. The RLC Layer	2
2.1.2. RLC Workload and Hardware-Managed Cache	3
2.2. CachePool	3
2.2.1. Compute Tile	4
2.2.2. Snitch-Spatz Core Complex	6
2.2.3. Baseline Memory Subsystem	8
2.3. HPDcache	9
2.3.1. HPDcache Architecture and Main Features	10
2.3.2. Configurability and Supported Operations	11
3. Related Work	12
4. Implementation	13
4.1. Cache Hierarchy	13
4.1.1. L1 Cache Organization	15
4.1.2. Spatz Request Coalescing	16
4.1.3. L1-Interconnect Interfacing	17
4.1.4. Memory Transaction Translation	18
4.2. Cache Coherence Design	19
4.2.1. Directory Organization	20
4.2.2. Coherence Protocol	21
4.2.3. Message and Data Channels	27
4.3. Cache Coherence Implementation	30
4.3.1. Directory Controller	32
4.3.2. Directory Bank Access	33
4.3.3. Coherence Message Network	34

Contents

5. Results	37
5.1. Evaluation Setup	37
5.1.1. Hardware Configurations	37
5.1.2. Testbenches and Software Workloads	38
5.2. Performance	38
5.2.1. Performance Impact Breakdown	40
5.3. Physical Design	41
5.3.1. Area	42
5.3.2. Timing	44
6. Conclusion and Future Work	45
A. Cachepool Hardware Configuration	46
B. HPDcache Configuration	48
C. Critical Path	50
D. Task Description	52
List of Acronyms	59
List of Figures	61
List of Tables	62
Bibliography	63

Introduction

Wireless baseband dataplane processing requires energy-efficient parallel computing to meet escalating throughput demands. This work focuses on the Radio Link Control (RLC) layer, which manages data queuing and assembly into Protocol Data Units (PDUs) to balance performance and reliability. As a computing workload, the RLC layer presents three primary architectural challenges:

- **Scale:** Modern network deployments are required to sustain approximately 10 million packets per second overall, with individual RLC instances processing over 10^3 packets per second across thousands of concurrent instances.
- **Latency:** The Transition Time Interval (TTI) sets a strict real-time envelope ranging from $62.5\ \mu\text{s}$ to 1 ms (typically $500\ \mu\text{s}$) for all sub-tasks.
- **Irregularity:** Tasks exhibit high variability and independence per user, featuring sparse, non-contiguous memory accesses over large address ranges due to data structures such as linked lists.

These properties result in a high number of concurrent threads accessing dispersed memory entries with poor spatial locality. Designs that explicitly manage memory, such as those relying on scratchpad memory (SPM) and Direct Memory Access (DMA), become inefficient because traditional techniques like tiling and double-buffering fail when data dependencies branch unpredictably.

To overcome these challenges, a hardware-managed, cache-based hierarchy is required. This project investigates "CachePool," a scalable, domain-specific, cache-coherent manycore architecture designed for dataplane workloads. The proposed architecture features private L1 caches per core complex (CC) utilizing a lazy write-through policy, backed by a logically shared, inclusive, and multi-banked L2 cache, alongside a lightweight hardware-managed coherence scheme. The primary objective is to sustain high-throughput processing within the strict TTI envelope while maintaining a light-weight memory model suitable for highly irregular and sparse access patterns.

Background

Before diving into the architectural and implementation details, the relevant theoretical and technical backgrounds need to be introduced. This section of the report will cover necessary background information, primarily concerning the RLC layer, CachePool architecture, and the HPDcache architecture.

2.1. Radio Access Network (RAN)

The fifth generation (5G) of wireless and mobile communications targets new services and business models, extending far beyond a standard evolution of 4G networks. The primary service types driving 5G architecture include extreme mobile broadband (xMBB) for data rates up to several gigabits per second, massive machine-type communications (mMTC) to connect millions of power-constrained sensors, and ultra-reliable low-latency communications (UMTC) [1]. This ultra-reliable class demands end-to-end latencies of less than 5 ms and a reliability of 99.999% to support critical applications such as vehicular communication [1].

To support these diverse and extreme requirements, the 5G RAN architecture must be designed to scale seamlessly in terms of throughput and the massive number of device connections [1]. The 5G RAN is envisioned to support a wide range of physical deployments, spanning from distributed base stations to highly centralized cloud-RAN configurations.

2.1.1. The RLC Layer

Within the RAN protocol stack, the Radio Link Control (RLC) layer operates as a critical, time-synchronous function [1]. In this dataplane layer, SDUs received from higher layers are queued and assembled into PDUs. These PDUs are equipped with sequence headers for retransmission management before being passed down to the Medium Access Control (MAC) layer. This specific interaction tightly couples the system's performance capabilities with its reliability constraints.

2. Background

Because RLC is a time-synchronous function, it demands high data rates on its network interfaces that scale directly with traffic, signal bandwidth, and antenna counts [1]. Processing the RLC layer, therefore, benefits significantly from, and often necessitates, hardware acceleration.

Viewed specifically as a computing workload, the RLC dataplane exhibits three defining features that complicate hardware and memory design:

- **Scale:** Modern RAN deployments must sustain an overall throughput of approximately 10 million packets per second. Furthermore, a system may have thousands of active instances running concurrently, with each individual RLC instance needing to exceed 1,000 packets per second.
- **Latency:** All sub-tasks within the RLC are bound by a hard real-time envelope set by the Transition Time Interval (TTI). This TTI spans from 62.5 μ s to 1 ms, with 500 μ s being typical, demanding strict real-time processing capabilities.
- **Irregularity:** Tasks performed on a per-user basis are independent but highly variable. While the actual code footprint for these tasks is small, the layer relies heavily on data structures such as linked lists. This creates highly sparse, non-contiguous memory accesses spread across large address ranges.

2.1.2. RLC Workload and Hardware-Managed Cache

The intersection of the RLC layer's scale, latency, and irregularity results in a processing environment where hundreds of concurrent threads must touch dispersed memory states exhibiting extremely poor spatial locality.

For such unpredictable access patterns, explicitly managed memory hierarchies, such as the combination of SPM and DMA, are no longer ideal choices. Common memory optimization techniques like tiling and double-buffering become highly inefficient because the underlying data dependencies branch in unpredictable ways.

To meet the real-time processing demands and timing requirements of synchronous RAN functions [1], a hardware-managed, cache-based memory hierarchy is required. This ensures that hundreds of Processing Elements (PEs) can efficiently access shared cache banks, providing the necessary hardware acceleration to sustain high-throughput processing within the strict TTI latency envelope.

2.2. CachePool

CachePool is a scalable, cache-based, heterogeneous manycore architecture specifically tailored for irregular dataplane workloads, such as the 5G RLC layer described in section 2.1.1 [2]. To effectively handle the unpredictable memory access patterns and strict latency constraints of RLC packet processing, the architecture shifts the focus away from traditional SPM approaches as well as complex global hardware cache coherence mechanisms. Instead, the CachePool architecture is built upon three foundational design principles:

2. Background

- **Shared L1¹ Cache with Configurable Private Partitions:** L1 cache is shared and accessible by all Processing Elements (PEs) across the cluster. However, software can dynamically reconfigure specific L1 partitions to act as private memory for a given tile. In this private mode, software completely manages cache coherence, which reduces average access latency, mitigates bandwidth contention, and improves timing predictability for heavily accessed data. [2]
- **Heterogeneous Scalar-Vector Execution:** The processing cluster pairs lightweight, in-order RISC-V scalar cores (Snitch) for control-intensive operations with tightly coupled vector engines (Spatz) for data-intensive processing [2]. This tight integration enables scalar and vector execution phases to interleave with near-zero switching overhead.
- **Scalable L1 Cache Controller:** To support the sparse, memory accesses characteristic of the RLC linked lists, CachePool utilizes a highly scalable L1 cache controller. This controller embeds Miss Status Holding Register (MSHR) metadata directly into the cache-line SRAM arrays [3]. This design eliminates the need for dedicated, fixed-size MSHR registers, allowing the architecture to sustain a massive number of in-flight cache misses [2].

2.2.1. Compute Tile

CachePool adopts a hierarchical organization in which the tile serves as the fundamental building block for scaling the manycore system. The tile is designed for replication and integrates a small number of processing elements together with local instruction, data, synchronization, and peripheral-access resources. In the current organization shown in Figure 2.1, each tile contains four 32-bit core complexes (CC0–CC3) [2]. This four-CC structure provides a regular and modular compute granularity, which can later be replicated to form larger clusters or multi-tile systems.

At the Tile level, the compute-side organization is centered around a shared local memory and interconnect structure. All four CCs connect to the tile memory system through a $5 \times \text{Tile}$ L1 crossbar, implemented as five independent 4×4 32-bit crossbars. The crossbar also incorporates bank interleaving and supports dynamic address remapping, allowing the tile to expose a flexible local data-access structure without pushing this complexity into the cache controller itself [2].

The tile further includes a shared L1 instruction cache and a banked shared L1 data cache structure. In the current tile microarchitecture, the instruction side is served by a shared 8KiB L1 instruction cache [2]. On the data side, the tile contains a 256KiB shared data cache, organized into four banks, each exposing a wide local interface [2]. This banked organization is a key tile-level design feature, as it enables multiple CCs to access local data resources concurrently and forms the basis for scalable bandwidth within the tile.

¹All references to the term 'L1 Cache' in Section 2.2 denote the existing cache system before any modification. The existing L1 cache will be the L2 cache when the private L1 cache is integrated.

2. Background

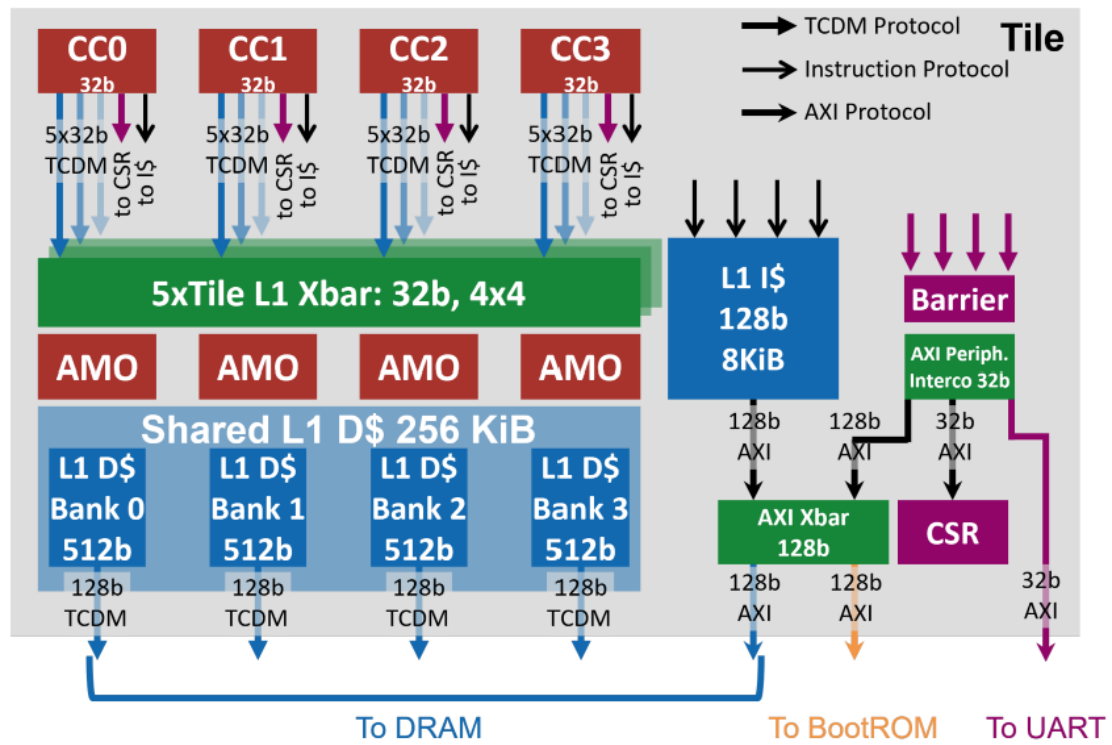


Figure 2.1.: Architecture of a CachePool compute tile before integrating private L1 caches [2].

2. Background

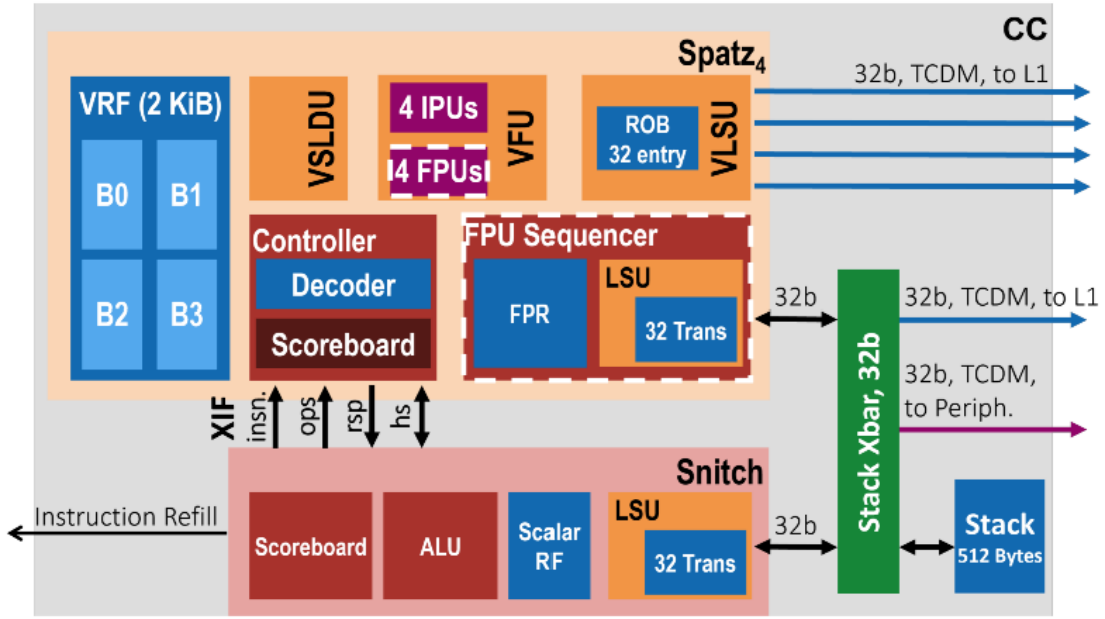


Figure 2.2.: Architecture of a CachePool Core-Complex (CC) [2].

Besides compute and memory access, the tile also integrates several supporting blocks. An AMO unit is placed on the path to the shared data to support RISC-V atomic memory operations, which are required for correct synchronization and ordering in a shared-memory multicore system [2]. Figure 2.1 also shows dedicated AXI-connected paths from the tile toward external system components such as DRAM, BootROM, and UART, illustrating the SoC-level connectivity.

Overall, the CachePool tile can be viewed as a modular shared-memory compute cluster: it groups four CCs, a shared local instruction path, a banked shared local data path, a hierarchical low-latency interconnect, and essential synchronization and peripheral-access logic into one replicable unit.

2.2.2. Snitch–Spatz Core Complex

Within a compute tile, the Core-Complex (CC) serves as the fundamental processing unit, designed as a tightly coupled, heterogeneous scalar-vector PE. As shown by Figure 2.2, each CC features a lightweight 32-bit RISC-V Snitch scalar core with a 32-bit Spatz vector engine as its computing unit [2]. This heterogeneity allows the architecture to efficiently balance control-heavy tasks with data-parallel workloads.

At a high level, the Snitch core acts as the host processor, handling standard scalar instructions and managing overall control flow [2]. When it encounters vector or floating-point instructions, it offloads them to the Spatz coprocessor through a dedicated accelerator port. This tight integration enables the CC to interleave scalar and vector execution phases.

2. Background

Externally, as illustrated in Figure 2.2, each CC interfaces with the broader cluster infrastructure through several distinct communication paths. For data operations, the CC utilizes a 5×32 -bit TCDM interface that connects directly to the shared L1 Data Cache [2]. Alongside this data path, it maintains dedicated connections to the L1 Instruction Cache and Control and Status Registers (CSR) [2].

Snitch Scalar Core

The Snitch core is an ultra-small and highly energy-efficient 32-bit RISC-V compute unit that serves as the foundational control processor within the system [4][2]. To maximize energy efficiency and maintain a minimal area footprint, the core is implemented as a single-stage, single-issue, in-order processor [4]. It implements the standard RV32I base instruction set architecture, with an alternative option to utilize the RV32E embedded profile to further minimize the size of the register file. The primary architectural objective of Snitch is to maximize the compute-to-control ratio, minimizing the area dedicated to control logic in order to pair the tiny core with a double-precision floating-point unit (FPU) [4].

To handle data-parallel tasks efficiently, Snitch is equipped with an accelerator of-flooding interface that features independent, decoupled request and response channels [4]. As shown in Figure 2.2, Snitch can offload vector workloads to Spatz Vector Core, which will be introduced in the following section. To mitigate variable memory access latencies, the core utilizes a non-blocking Load Store Unit (LSU). This LSU is capable of managing a configurable number of outstanding load requests to the memory hierarchy and executing RISC-V standard atomic memory operations [4].

Spatz Vector Core

Spatz is a compact 32-bit vector processing unit based on the RISC-V Vector Extension (RVV) version 1.0, targeting the Zve32x subset for embedded vector machines [5]. It is designed as a small and energy-efficient vector engine that can be coupled to a lightweight scalar processor through a generic accelerator interface. As shown in Figure 2.3, Spatz is built around a centralized latch-based Vector Register File (VRF), which is divided into four banks and serves as the operand storage for its vector datapath [5]. This centralized VRF is a key design choice, as it simplifies the implementation of vector permutation operations and supports efficient vector chaining while maintaining a compact hardware structure.

The execution backend of Spatz is composed of three main functional units as shown by the three orange boxes on the right side of Figure 2.3:

- **Vector Load/Store Unit (VLSU):** It is responsible for all vector memory operations. It supports unit-stride and constant-stride access patterns and interfaces with the memory system through multiple narrow ports to sustain bandwidth. The VLSU also incorporates a reorder buffer to reassemble data before committing [5].

2. Background

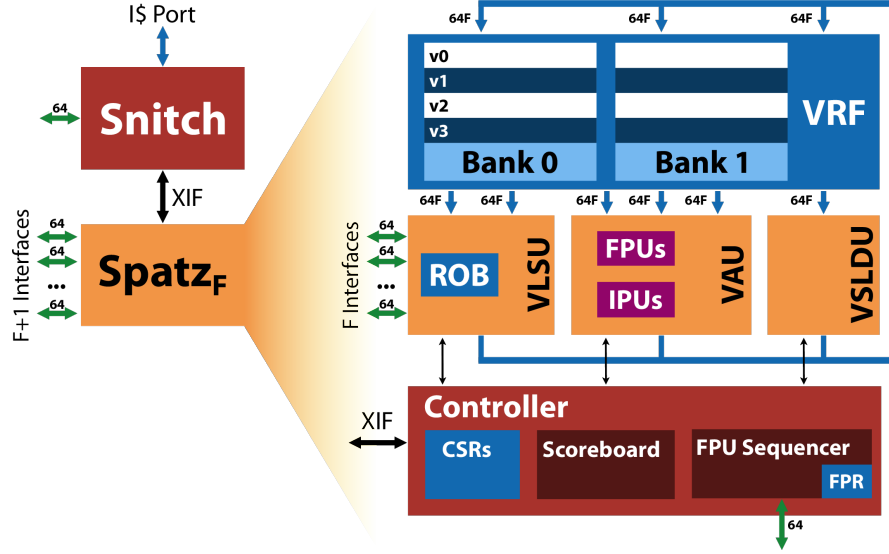


Figure 2.3.: Architecture of Spatz vector engine[5].

- **Vector Arithmetic Unit (VAU):** The VAU performs the core vector computations. It consists of a set of parallel arithmetic lanes, each equipped with multiply-accumulate units capable of operating on 8-bit, 16-bit, and 32-bit integer elements [5]. The VAU is designed to exploit data-level parallelism by executing the same operation across multiple vector elements simultaneously [5].
- **Vector Slide Unit (VSLDU):** The VSLDU handles vector permutation operations, such as element shifts, slides, and data rearrangements [5]. These operations are essential for many algorithms that require alignment or reorganization of data within vectors. By providing dedicated hardware support for such transformations, the VSLDU avoids inefficient sequences of scalar or arithmetic instructions and improves overall execution efficiency.

These three units operate under the control of the Spatz' controller (red box on the bottom-right of Figure 2.3), which decodes vector instructions, manages the RVV control and status registers, tracks in-flight operations through a scoreboard, and enables fine-grained vector chaining [5].

In CachePool, Spatz is used as the vector engine within each CC, where it complements the lightweight scalar core by accelerating data-parallel portions of the workload. This scalar-vector organization aligns with the project goal of providing scalar-vector capability for dataplane processing in a scalable manycore architecture.

2.2.3. Baseline Memory Subsystem

CachePool adopts a single-level shared cache organization at the tile level before the integration of the private L1. Each tile contains a shared 256KiB four-banked L1 data

2. Background

cache, which is accessed by the four CCs through a TCDM-based hierarchical crossbar, while instruction fetch is served by a shared 8KiB L1 instruction cache [2]. In this baseline organization, the shared L1 data cache is the main local memory structure of the tile, and there is no intermediate L2 cache. This design comes at the cost of a more demanding on-tile interconnect [2]. The tile-level data path is organized around banked shared caching. The four banks of the L1 data cache are connected to the CCs through a TCDM crossbar, with bank interleaving and lightweight address scrambling handled in the interconnect [2].

Insitu-Cache

The baseline L1 data cache is managed by the InSitu Cache controller. InSitu Cache is a non-blocking cache design that tolerates long-latency accesses directly to DRAM, while avoiding extra SRAM structures such as dedicated write buffers or conventional miss handling architectures [3]. Its miss handling reuses cache resources, including unified in-cacheline handling for read and write misses, Dynamic Subentry Expansion (DSE) to increase effective miss capacity under long memory latency, and a Dynamic Subset LRU (DS-LRU) policy to reclaim otherwise unused cache lines for miss tracking [3]. These properties make it suitable for a shared cache directly exposed to variable-latency off-chip memory.

DRAMSys

Off-chip memory is modeled through DRAMSys in co-simulation. The CachePool work reuses infrastructure developed in TeraPool [6], where an RTL interface connects the cluster AXI interconnect to DRAMSys compiled as a shared library and linked into the simulation flow. In CachePool, this setup is reused with a default DDR4 four-channel configuration [2]. More generally, DRAMSys is a cycle-accurate SystemC/TLM-based DRAM simulation framework that models an arbiter plus per-channel controllers and DRAM devices, enabling accurate exploration of bandwidth and latency effects without the prohibitive cost of full low-level simulation [7].

2.3. HPDcache

HPDcache is an open-source high-performance cache for RISC-V cores, developed to address the memory bottleneck caused by limited concurrency in existing open-source cache hierarchies. It is implemented in SystemVerilog and released through the OpenHW ecosystem. Compared with more conventional caches, HPDcache is designed as a non-blocking, pipelined, and highly configurable cache that can sustain multiple outstanding transactions while maintaining a simple integration model for processor cores and tightly coupled accelerators [8].

In this work, HPDcache is adopted as the per-CC private cache for CachePool. Its open-source availability, flexible interface structure, non-blocking design, and sup-

2. Background

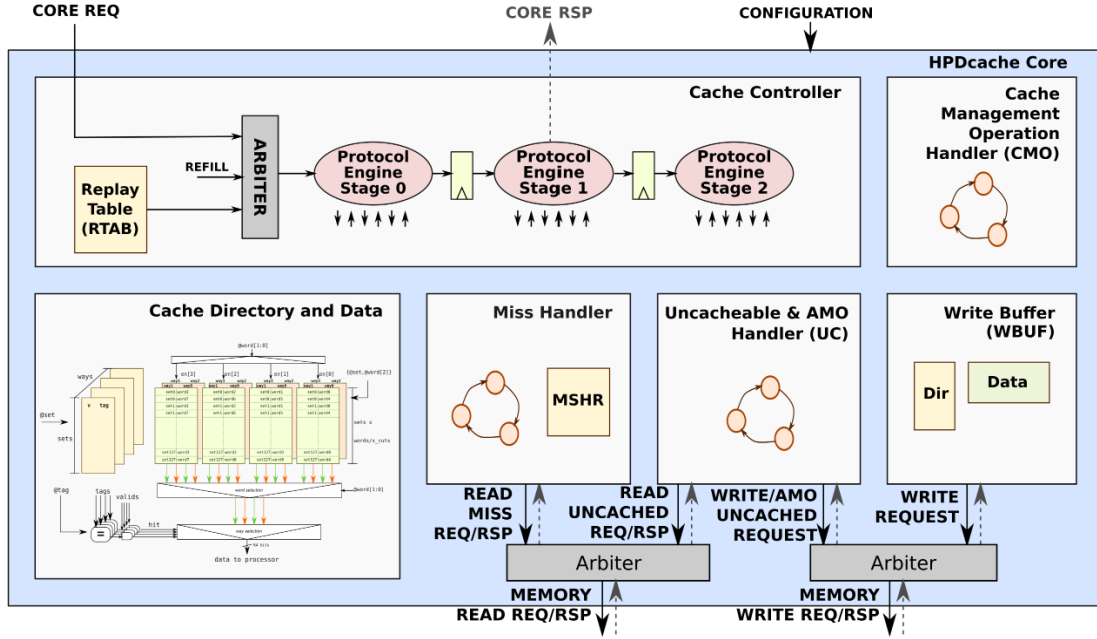


Figure 2.4.: HPDcache Core [9].

port for write-through operation make it a natural candidate for extension toward the CachePool memory hierarchy.

2.3.1. HPDcache Architecture and Main Features

As illustrated in Figure 2.4, HPDcache consists of a cache controller, cache directory and data arrays, a miss handling structure, a replay mechanism, and a write buffer [8]. The cache implements a three-stage pipeline and can accept one request per cycle. It supports multiple requesters, including core load/store units, accelerators, and an optional hardware prefetcher [8]. The request and memory interfaces use ready-valid channels, and the memory side can be adapted to AXI through a provided adapter. HPDcache also supports a wide range of memory interface widths, from 64 bits up to 512 bits, which allows it to be tailored to different bandwidth requirements [9].

A key property of HPDcache is that it is non-blocking. Rather than stalling the processor on every cache miss, it can maintain multiple outstanding read misses through a configurable MSHR. The MSHR can be implemented in SRAM rather than flip-flops, which enables a much deeper structure with lower area cost[8]. This is especially important for workloads with irregular accesses, where several misses may be in flight simultaneously.

HPDcache also includes a write buffer to support its write-through operation. In the write-through mode, the write buffer hides response latency and performs write coalescing, merging adjacent writes to reduce memory traffic [8]. HPDcache also support hybrid write-through/write-back capabilities and dynamic write-policy hint.

2. Background

2.3.2. Configurability and Supported Operations

Another important feature of HPDcache is its high degree of configurability. The cache exposes more than 25 synthesis-time parameters, covering the number of requesters, cache geometry, MSHR dimensions, replay table depth, write-buffer sizing, memory interface width, and optional support for AMOs, CMOs, and performance counters[10]. This flexibility makes the design usable both in high-performance cores and in smaller embedded systems.

Functionally, HPDcache embeds native supports on atomic memory operations (AMOs) and cache management operations (CMOs). The latter include operations such as memory write fences, cache-line invalidation, invalidation of the full cache, and explicit prefetching[9]. It also provides a CSR space for event and performance monitoring. These features are useful in multi-core and heterogeneous systems, where visible control over the cache and correct synchronization behavior are important.

Related Work

A large body of prior work has studied cache coherence for shared-memory multicore and manycore systems. In conventional designs, directory-based MESI/MOESI-style protocols are widely used because they avoid the broadcast cost of snooping and scale better with increasing core count. At the same time, their complexity is substantial: directories must track sharers, invalidations must be coordinated carefully, and a large number of stable and transient states are often required. Ros and Kaxiras argue that much of this complexity comes from trying to provide transparent coherence under strong memory-model assumptions, and explore simpler alternatives that trade protocol aggressiveness for lower hardware cost[11].

Among write-through, directory-based approaches, the DHCCP protocol used in the TSAR architecture is particularly relevant to this work. DHCCP places a distributed directory alongside the inclusive L2 cache and uses a write-through policy at L1 to simplify coherence[12]. For small sharer counts, it keeps explicit copy locations and applies multicast update; for larger sharer counts, it falls back to broadcast invalidation[12]. It also separates direct and coherence transactions across distinct networks to avoid deadlock. This makes DHCCP a useful reference point for scalable NoC-based coherence, especially for architectures with distributed shared lower-level caches.

Several works have attempted to reduce the cost of write-through coherence. RWT extends DHCCP with a hybrid policy that keeps shared lines coherent using write-through, but allows non-shared lines to use write-back, with the coherent/non-coherent decision made in hardware at cache-line granularity[13]. This reduces write traffic while preserving the simplicity of write-through for shared data.

Other lines of work simplify coherence by reducing invalidation traffic or even removing the directory. SARC introduces tear-off cache blocks and writer prediction to reduce invalidation traffic[14], while Complexity-Effective Multicore Coherence combines dynamic write policy with self-invalidation at synchronization points to eliminate the directory entirely for data-race-free programs[11]. These approaches are attractive from a complexity and power perspective, but they rely on weaker assumptions.

Implementation

This chapter presents the architectural design and implementation details of the extensions to the cache hierarchy of CachePool developed in this work. Building upon the background introduced in Chapter 2, this chapter will describe the key design choices, implementation approach, and technical considerations behind the integration of the new cache hierarchy and its supporting mechanisms.

The chapter is organized as follows. It first introduces the design of the private L1 data cache adopted for each CC. Then, it presents the cache coherence mechanism developed to maintain consistency across the multi-core system. Finally, it describes the back-end work, including the physical-design considerations and efforts associated with the proposed architecture.

4.1. Cache Hierarchy

Aiming to address the limitations of the baseline tile organization, CachePool introduces a two-level cache hierarchy in which each CC is equipped with a private L1 data cache, while the original shared tile-level L1 data cache is repurposed as a logically shared, inclusive, multi-banked L2 cache. Figure 4.1 illustrates a simplified architectural view of a CachePool tile with the key components. Each tile contains four 16KiB L1 data caches implemented with HPDcache and private to each CC, connected through a crossbar cache interconnect to four shared L2 banks implemented using InSitu-Cache. This reorganization follows the design purpose to better support irregular dataplane workloads while preserving a shared-memory programming model.

Compared with the baseline design, where all CCs directly access a fully shared banked data cache, the new hierarchy moves frequently reused and latency-sensitive data closer to each CC and reduces contention on the shared tile-level memory structure. Inclusion is maintained between the two levels, which means that any line present in an L1 cache must also reside in L2. This property is important for the coherence design presented later in Section 4.2, since it allows the shared L2 to act as the point of coordination for cache-line ownership and sharer tracking without requiring a separate

4. Implementation

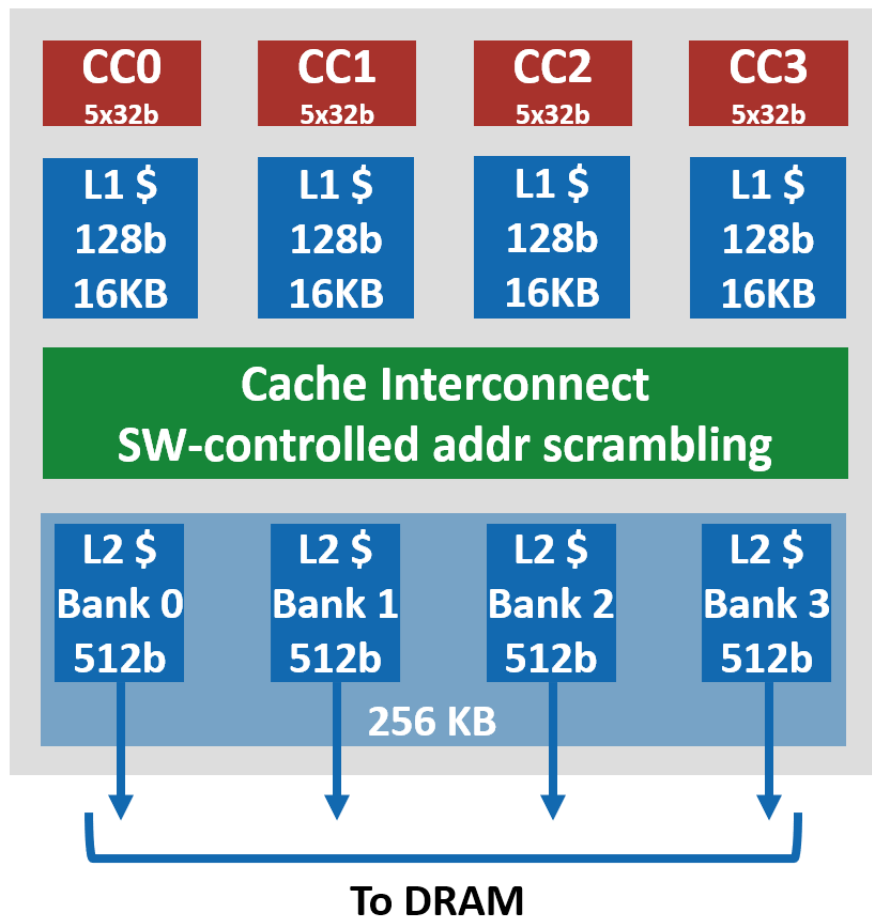


Figure 4.1.: Architecture of a CachePool Tile with Private L1 Cache.

4. Implementation

Table 4.1.: Key configuration parameters of the private L1 data cache.

Parameter	Value
Number of sets	64
Associativity	4-way set-associative
Words per cache line	4
Word width	128 bit
Cache line size	512 bit (64 B)
Total cache size	16 KiB
Number of caches per tile	4

directory structure. The AMO processing unit between the interconnect and the data cache in Figure 2.1 has been removed as AMO operations are handled by internal mechanism of HPDcache.

From an implementation perspective, the new hierarchy reuses the existing shared-cache infrastructure while inserting a private cache level above it for each CC. The private L1 is built on HPDcache, a non-blocking write-through L1 cache with multiple outstanding misses, out-of-order request replay, and a wide configurable memory interface, making it well suited to the irregular and memory-intensive behavior of the target workload [8]. The shared lower level continues to rely on InSitu-Cache, which already serves as the tile-level shared cache in the baseline architecture [2].

The cache interconnect, shown as the green rectangular box in Figure 4.1, retains the same bank interleaving mechanism as in the baseline design. In particular, L2 bank selection is performed inside the crossbar together with a software-controlled dynamic address scrambling scheme [2]. This preserves compatibility with the original tile-level memory organization and keeps the lower-level bank mapping flexible. As a result, the shared L2 banks can still be distributed and mapped through software-visible control, while the private L1 caches remain agnostic to the exact physical bank placement behind the interconnect.

4.1.1. L1 Cache Organization

Each CC is equipped with a private L1 data cache based on HPDcache. Table 4.1 shows the key configurations of the L1 cache. The cache is configured as a 4-way set-associative structure with 64 sets. Each cache line consists of 4 words, and each word is 128 bit wide, resulting in a cache-line size of 512 bit (64B). With this organization, the total capacity of each private L1 cache is 16 KiB.

The choice of a 128 bit cache word width is primarily motivated by the data width used by Spatz. In the current CC, Spatz vector engine handles memory operations with a continuous 4×32 bit pattern (see Section 4.1.2), which naturally maps to a 128-bit word with. Matching the cache word width to this vector width simplifies the data transfer between the private L1 cache and the CC, and avoids unnecessary fragmentation of a vector word across multiple narrower cache words.

4. Implementation

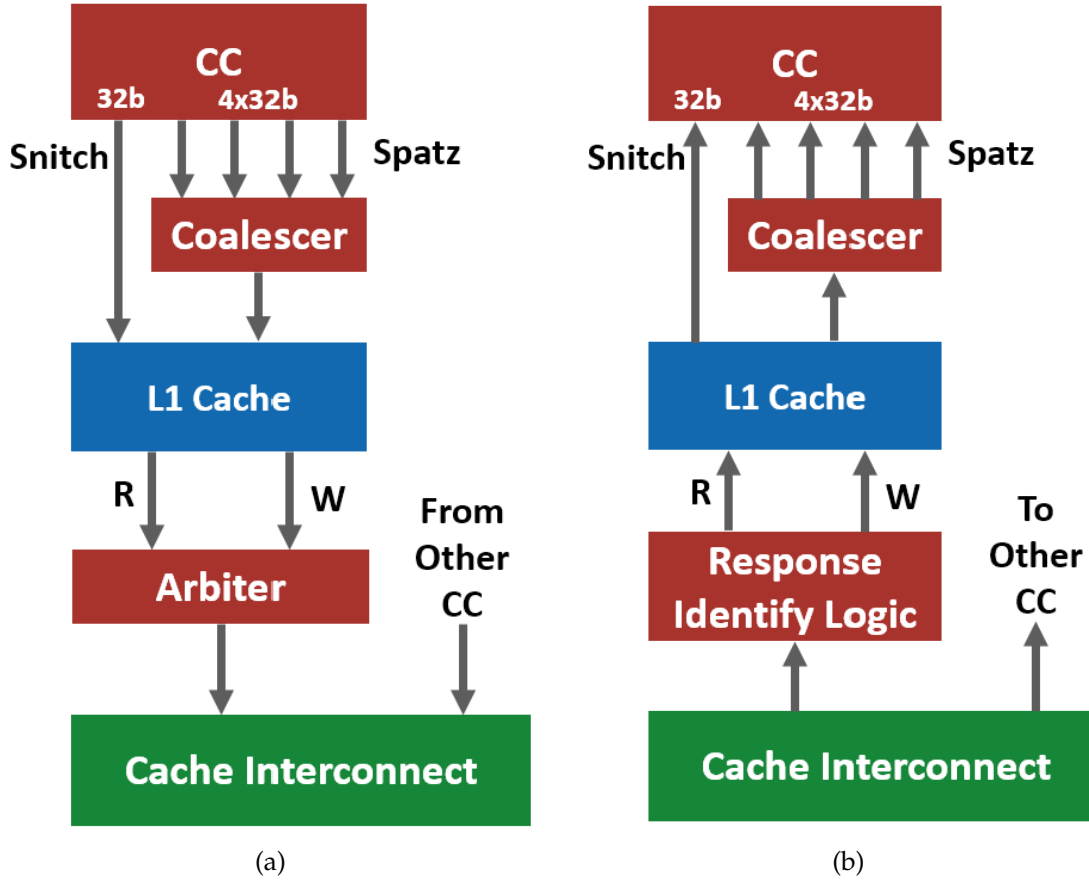


Figure 4.2.: (a) Request and (b) Response Datapath of L1 cache.

The selected configuration provides a balanced trade-off between capacity, associativity, and implementation complexity. A 4-way set-associative structure reduces conflict misses compared with a direct-mapped or low-associativity design, while still keeping the tag and data lookup overhead moderate. Meanwhile, the 64 B cache-line size is large enough to exploit spatial locality for contiguous accesses, yet not excessively large for the irregular workloads targeted by CachePool.

Overall, this cache organization is chosen to provide a compact private L1 close to each CC, while aligning its internal data granularity with the execution characteristics of the attached scalar-vector core complex.

4.1.2. Spatz Request Coalescing

As Figure 4.1 shows, each CachePool CC exposes five 32-bit TCDM request/response ports toward the memory system. Figure 4.2 shows their composition: one 32-bit port from Snitch and four 32-bit ports from Spatz. This organization follows the native memory behavior of Spatz, whose VLSU uses a parametric number of 32-bit memory

4. Implementation

interfaces in order to match the vector engine datapath [5]. In the baseline shared-memory organization, these narrow and independent interfaces are handled by the cache controller of the shared data cache.

After introducing a private HPDcache-based L1 cache per CC, the on-tile datapath is adapted to better match the cache granularity. HPDcache supports a configurable number of requesters, and in this work it is configured with two request ports: one for Snitch and one for Spatz. HPDcache also supports wide requester data channels, with the requester-side width determined by the cache configuration. This makes it possible to present the vector-side traffic to the cache as a single wider access instead of multiple fragmented narrow accesses.

Figure 4.2a is a simplified illustration of the request datapath between a CC and the cache interconnect. Before feeding to L1 cache, the four 32-bit memory requests generated by Spatz are passed through a coalescer. Since the requests naturally correspond to continuous vector data, they can be effortlessly merged into one 128-bit cache access with a continuous address range. With coalescing, L1 cache consumes the four Spatz memory operations as a single request in the same cycle, rather than as four separate 32-bit transactions.

The Snitch memory port is treated differently. Its traffic is scalar and 32-bit wide, and it does not align with the four-lane vector access pattern of Spatz. Since the Snitch request typically differs largely from the vector data accessed by Spatz, an attempt for it to join the coalescing process is difficult and meaningless. The Snitch request bypasses the coalescer and connects directly to the second requester port of HPDcache. From the cache point of view, the private L1 therefore serves two distinct traffic ports.

On the response path as shown in the upper part of Figure 4.2b, the reverse operation is performed. The 128-bit cache word is decomposed by the same coalescer and delivered to the four response channels of Spatz whereas the fifth channel bypasses and returned directly to Snitch. In this way, the external CC-side interface remains compatible with the original five-port organization, while the cache-side interface is reduced to a cleaner and more efficient two-requester structure.

4.1.3. L1-Interconnect Interfacing

After the private L1 cache is inserted between the CC and the shared lower memory hierarchy, an interface adaptation layer is required between HPDcache and the CachePool tile interconnect. The reason is that the two sides use different protocols. On the cache side, HPDcache exposes its memory interface as a 5-channel ready-valid interface, resembling that of AMBA-AXI interface [9][15]. It features separate channels for read requests, read responses, write requests, write data, and write responses. This interface allows the downstream traffics of HPDcache to be easily adaptable to common NoC protocols such as AXI, but it does not directly match the protocol used for the CachePool cache interconnect.

In CachePool, the tile-level interconnect is based on TCDM. This choice is motivated by physical feasibility and low-latency on-tile communication. In comparison with AXI, the TCDM protocol has a simpler channel structure and lower wiring complexity, which

4. Implementation

makes it more suitable for dense cluster-level memory access [2].

In context of this work, the main mismatch between the two protocols are the number of ports: the TCDM-based cache interconnect does not accept read and write requests with separate channels, nor does it provide an independent channel for write data as AXI does. Memory traffic must be presented as a single request stream toward the interconnect and lower cache hierarchy. To bridge this mismatch, an arbiter is inserted on the request path between the L1 cache and the cache interconnect as shown in Figure 4.2a, selecting between the downstream read-request and the write-request (including write data) from the L1 cache and pushes the chosen transaction through the interconnect. In this way, the split memory-side interface of HPDcache is serialized into the request format expected by the network and L2 cache.

The reverse operation is required on the response path. Responses returning from the cache interconnect may correspond either to read data for a previous miss request or to acknowledgements for a write request. Since HPDcache expects these responses to arrive on different input channels, a Response Identify Logic block is placed between the interconnect and the cache as illustrated in Figure 4.2b. This block demultiplexes the upstream transactions by determining the type of the responses, and routes them to the appropriate L1 cache response port. Functionally, it acts as the inverse of the request-side arbiter.

This interfacing scheme allows the private L1 to be integrated into the existing CachePool memory system without modifying the fundamental TCDM-based tile interconnect. At the same time, it preserves the native HPDcache memory interface on the cache side, avoiding intrusive changes to the cache internals. The resulting design keeps the memory traffics compatible with the CachePool on-tile standard, while providing a clean protocol boundary around each private L1 cache.

4.1.4. Memory Transaction Translation

To preserve a clean protocol boundary, translation logics are inserted on both sides of HPDcache. On the upstream side, incoming TCDM requests are converted into HPDcache request messages before being pushed to the cache. On the downstream side, HPDcache memory requests are translated back into TCDM transactions before entering the arbiter and the interconnect crossbar. The response path follows the same principle in the opposite direction.

A key issue in this translation is the handling of transaction metadata. TCDM messages in CachePool carry several customizable metadata fields that HPDcache request and response flits do not natively provide dedicated fields for. To solve this, the metadata is packed into the HPDcache transaction identifier `hpdcache_req.tid` when the requests are translated. The encoded fields are:

- `wordOffset`: word selection offset
- `req.user.is_fpu`: FPU flag
- `req.user.core_id`: core ID

4. Implementation

- `req.write`: write flag
- `req.user.req_id`: TCDM request ID

The `user.is_amo` flag is not packed as a part of it because the AMOs are handled by the HPDcache as mentioned in Section 4.1.

This approach keeps HPDcache integration minimally intrusive. The `tid` field is internally reconstructed by HPDcache when returning the response, and further unpacking allows the recovery of TCDM metadata for in-order delivery of responses to the requestor [2]. As a result, the surrounding CachePool infrastructure can continue to operate with the efficient, scalable, and flexible TCDM semantics, while HPDcache gets connected through a well-defined adaptation layer.

4.2. Cache Coherence Design

With the introduction of private L1 data caches at each core complex, CachePool no longer has a single shared on-tile data cache as the only coherence point. The same memory block may now be replicated in multiple L1 caches, and these copies may be read and written by different CCs at different times. In such a system, a cache coherence protocol must exist to ensure all caches observe a consistent value when accessing the memory content. This issue is particularly important here because this work moves from the baseline shared-cache organization to a hierarchy with private L1 cache.

At a high level, replicating data in private L1s can serve local reads quickly with a shorter access path and reduce pressure on the shared lower level, but it also brings the risk of inconsistency. The system must clearly define the validity of copies and how concurrent requests are serialized. In CachePool, these questions are further shaped by the write-through nature of the private cache design and by the inclusive shared L2. Inclusion guarantees that every valid L1 line is also present in L2, which makes the L2 a natural coordination point for managing cache coherence.

Directory-based protocol is chosen rather than a snooping protocol for its scalability. Traditional snooping protocols rely on frequent broadcasting, and every coherence controller must observe and process every request. This is manageable in small systems, but the cost and pressure can easily grow beyond the acceptable level as the number of caches, as well as the size of interconnection, grows [16]. In contrast, a directory protocol keeps an explicit summary of the coherence state for each cache line. Broadcasts in snooping protocols are replaced with targeted communication, making directory-based protocol much better suited for CachePool, a hierarchical architecture designed to scale beyond a single tile [2].

The remainder of this section presents the proposed coherence protocol in detail. It first introduces the directory organization (Section 4.2.1), the main coherence operations and state transitions (Section 4.2.2), and message channels (Section 4.2.3).

4. Implementation

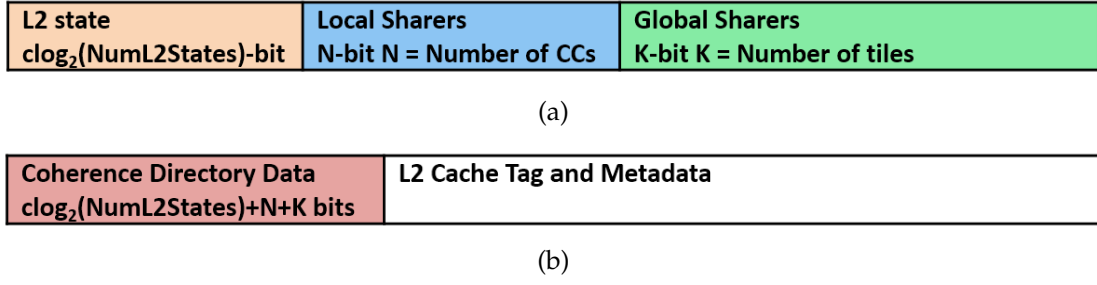


Figure 4.3.: (a) Coherence Meta Data Stored in Directory and (b) Extended L2 Tag Line.

4.2.1. Directory Organization

The coherence directory is placed at the shared L2 level, with one directory controller per L2 bank. This design choice follows the concept of an inclusive directory cache embedded in an inclusive last-level cache (LLC) presented in [16]. The key observation is that directory information only needs to be maintained for cache lines that may actually reside in on-chip caches [16]. For a semantically inclusive multi-level cache, any block present in a higher-hierarchy private cache (e.g. CachePool L1 cache) must also be present in the LLC (i.e. shared L2 cache). Therefore, a miss in the inclusive LLC immediately implies that the block is not cached on chip and is therefore in the invalid state in all private caches [16]. This allows the directory to be embedded directly in the LLC by an extension on its tag or directory, rather than maintaining a separate directory structure.

The CachePool coherence design adopts this idea at the L2 level. Since the proposed hierarchy uses private L1 caches backed by a logically shared, inclusive L2, the L2 is the natural point at which directory information can be maintained. The L2 tag bank can be extended with a few per-line directory bits instead of introducing a separate coherence directory.

In the proposed organization, the existing metabank of L2 cache, which already stores the cache-line tag and metadata [3], is extended to include coherence information as illustrated in Figure 4.3b. As shown in Figure 4.3a, each L2 cache line carries three classes of directory information:

- **L2 coherence state (The orange part in Figure 4.3a):** Logs the current state (MESI) of the cache line in L2. The size of this part is $\lceil \log(\text{NumL2State}) \rceil = \lceil \log(4 + 1) \rceil = 3$ bits. Details on coherence states are presented in Section 4.2.2.
- **Local sharer vector (The blue part in Figure 4.3a):** Fine-grained vector that tracks the CC-level ownership of a cacheline as a coherency node. The size of this part is equivalent to the number of CCs on a tile. Each bit represents the sharing state of a CC with 1 being a sharer and 0 otherwise.
- **Global sharer vector (The green part in Figure 4.3a):** Coarse-grained vector dedicated for multi-tile scaling. In a similar way to the local sharer vector, it uses

4. Implementation

Cache Line State $\text{clog}_2(\text{NumL1States})$ bits	L1 Cache Tag and Metadata
--	---------------------------

Figure 4.4.: Extended Tag Line of L1 cache.

one bit per tile and records whether the line may be cached in another tile.

For requests that remain within a tile, the local directory bits are sufficient to determine the set of sharers and to carry out the required coherence action. When a coherence transaction involves another tile, the request is sent to the directory controller of the destination tile. That controller then resolves the operation using its own fine-grain local sharer state and performs the corresponding intra-tile coherency update. This organization avoids the need for a globally centralized fine-grain directory, while still allowing the protocol to scale beyond a single tile.

Within a tile, fine-grain CC-level tracking is affordable because the number of private L1 caches is small, and it allows invalidations or ownership changes to be directed exactly to the relevant on-tile caches. Across tiles, however, maintaining a complete CC-level sharer list for the whole system would scale poorly.

Overall, the directory organization follows the principle of exploiting inclusion and multi-granularity design to provide a foundation for a scalable lightweight MESI-based protocol.

L1 Coherence Representation

The private L1 cache manages its coherence state in a similar manner by extending each cache-line tag entry with additional coherence bits. In contrast to the L2 directory, the information stored at L1 is much simpler. Since sharer tracking and ownership management are centralized at the inclusive L2, the L1 cache does not maintain any sharer information of its own. Each L1 cache line therefore only stores its local coherence state alongside the original tag and cache metadata, as illustrated in Figure 4.4. Details on L1 coherence states will be explained in Section 4.2.2.

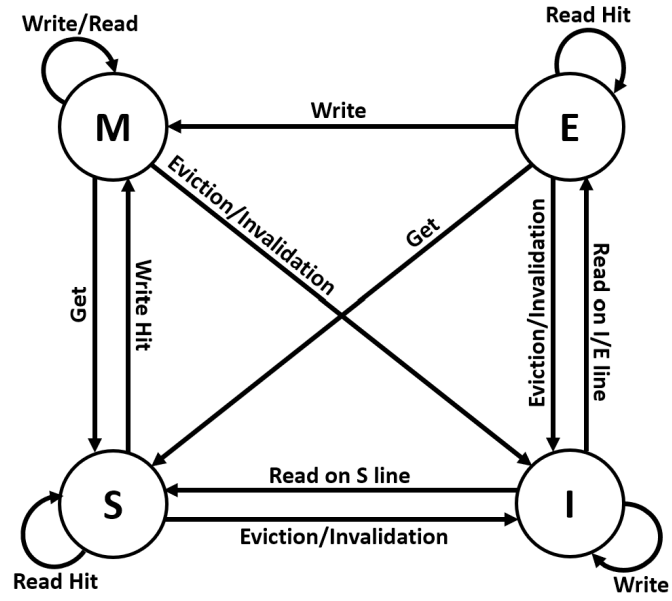
4.2.2. Coherence Protocol

The proposed coherence protocol is based on the four stable states of the MESI model: **Modified (M)**, **Exclusive (E)**, **Shared (S)**, and **Invalid (I)** [16]. These states describe the coherence status of a cache line and define what kind of access permission a cache currently holds.

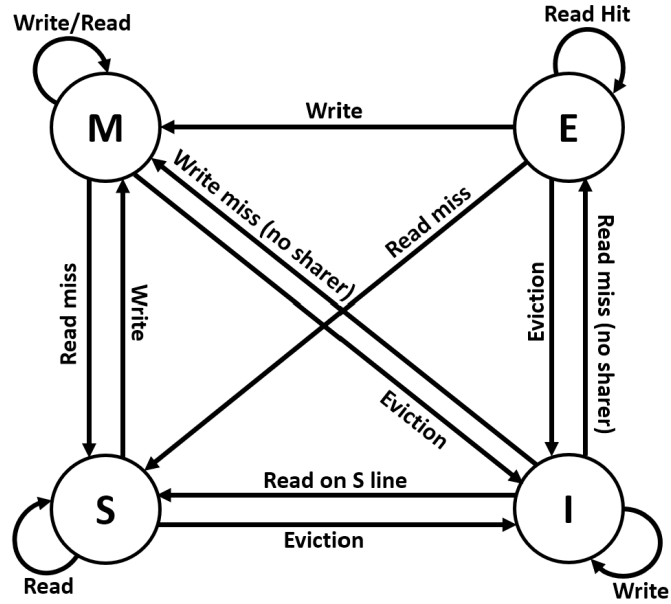
In general, the four state holds the following meaning:

- **I State:** the cache line is not present or not usable in the local cache.
- **S State:** line may be replicated in multiple caches and is read-only.
- **M State:** cache holds the latest writable copy of the line.

4. Implementation



(a)



(b)

Figure 4.5.: Top-level state transition FSM for a cache line in (a) L1 cache and (b) L2 directory controller.

4. Implementation

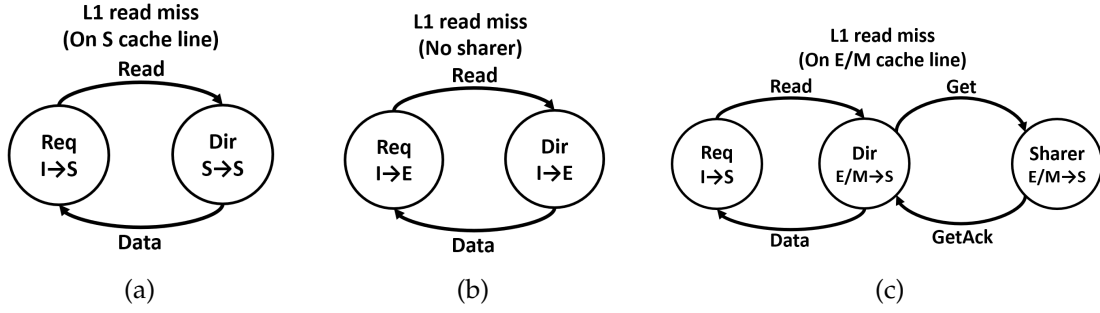


Figure 4.6.: Coherence State Transition when L1 Read Miss on (a) S Cache Line (b) I Cache Line, and (c) E/M Cache Line.

- **E State:** line is present in exactly one cache and may be updated locally.

The MESI model is tailored to the inclusive write-through cache model of CachePool. The S and I states retain their usual meaning. The E state is useful as it allows reads and subsequent writes to proceed without sending unnecessary coherence operations. The M state becomes special in a write-through design. The practical difference between M and E is smaller than in a conventional write-back cache. Nevertheless, retaining M remains meaningful, especially when write buffering and the later extension toward lazy write-through behavior are considered.

Figure 4.5 illustrates the top-level FSM for a cache line in L1 cache (Fig. 4.5a) and L2 directory controller (Fig. 4.5b). In the remainder of this section, the coherence protocol will be introduced in the following order: read hit, read miss, write to I/S cache line (both miss and hit), write to E/M cache line, and eviction.

L1 Read Hit

On an L1 read hit, the access is completed entirely within the private cache, and the cache-line coherence state remains unchanged. Since the access does not modify ownership or permissions, the L2 cache and the directory controller are not involved. This is the simplest coherence scenario and represents the normal low-latency hot cache read.

L1 Read Miss

On an L1 read miss, the request is forwarded to the L2 directory controller, which determines the coherence action according to the current directory state of the target line. As shown in Figure 4.6, there are three main cases.

The simplest case is a read miss on an S line shown in Figure 4.6a. Here, the directory already records the line in state S, meaning that the cache line is clean and no exclusive owner exists. The directory state remains S and updates the sharer information to include the new requester. Once the data arrives at the requester L1 cache, it may perform the I-to-S transition.

4. Implementation

The second case is a read miss on an I line with no sharer. In this situation, the line is not cached by any L1, so the requester becomes the sole owner of the line after the refill. Therefore, both the requester and the directory transition from I to E, as illustrated in Figure 4.6b. A read to an unshared line no longer causes an I-to-S transition; instead, it directly obtains the line in Exclusive state. This avoids unnecessarily classifying a privately held clean line as shared and reduces later coherence traffic if the same CC writes the line.

The third case is a read miss on a line currently held in E or M by another L1 (Figure 4.6c). In this case, the requester cannot be served directly from L2 without first resolving the exclusive ownership. Due to the existence of the write buffer and network latency, the directory may not necessarily hold the latest state of this exclusively-owned cache line. Therefore, a GET message is sent to the owner to probe for the latest copy and state of the cache line. The owner responds with GetAck, providing the most recent state information and flushing its write buffer if necessary. After receiving this acknowledgement, L2 sends the up-to-date data to the requester. For the L1 caches, the requester may upgrade the state from I to S, and the previous owner will have to downgrade the cache line to S after receiving a GET. At the directory level, the line transitions from E/M to S, and the new requester will be added to the sharer list.

Write to I/S Cache Line (Miss and Hit)

For a write to an I or S cache line, the protocol distinguishes three cases, as shown in Figure 4.7. In all cases, the L2 directory is informed as soon as the write request is issued, because a write affects global ownership and sharer information regardless of hit or miss.

The first case is a write to an invalid line with no existing sharer, shown in Figure 4.7a. Since the private L1 uses a non-allocate write-through policy [9], it does not allocate the line on a write miss. For this reason, an I-to-M transition does not occur at L1-level. The write is sent directly to L2, and the directory transitions from I to M because the system now has a single writer for that line. In this case no invalidation is required.

The second case is a write miss on a cache line with sharers or owner (S/E/M), shown in Figure 4.7b. For the same reason as the previous case, the cache line remains in I at L1. At the directory, the write request triggers a transition to M. To establish exclusive write permission, the directory updates its sharer information and sends invalidation to all other sharers of the line.

The third case is a write hit on a shared line, shown in Figure 4.7c. Here, the requester already holds the line in S and wants to upgrade it to the M state. The local L1 therefore transitions from S to M, while the directory performs the same transition. As in the previous case, the directory sends invalidation to all other sharers. The invalidation receivers send InvAck to the requester L1, and the write cannot be committed until all required invalidation acknowledgments have been received. To support this synchronization, the directory informs the requester of the number of invalidation acknowledgments expected. Once the expected number of InvAcks has arrived, the requester gains exclusive ownership, and the write may proceed.

4. Implementation

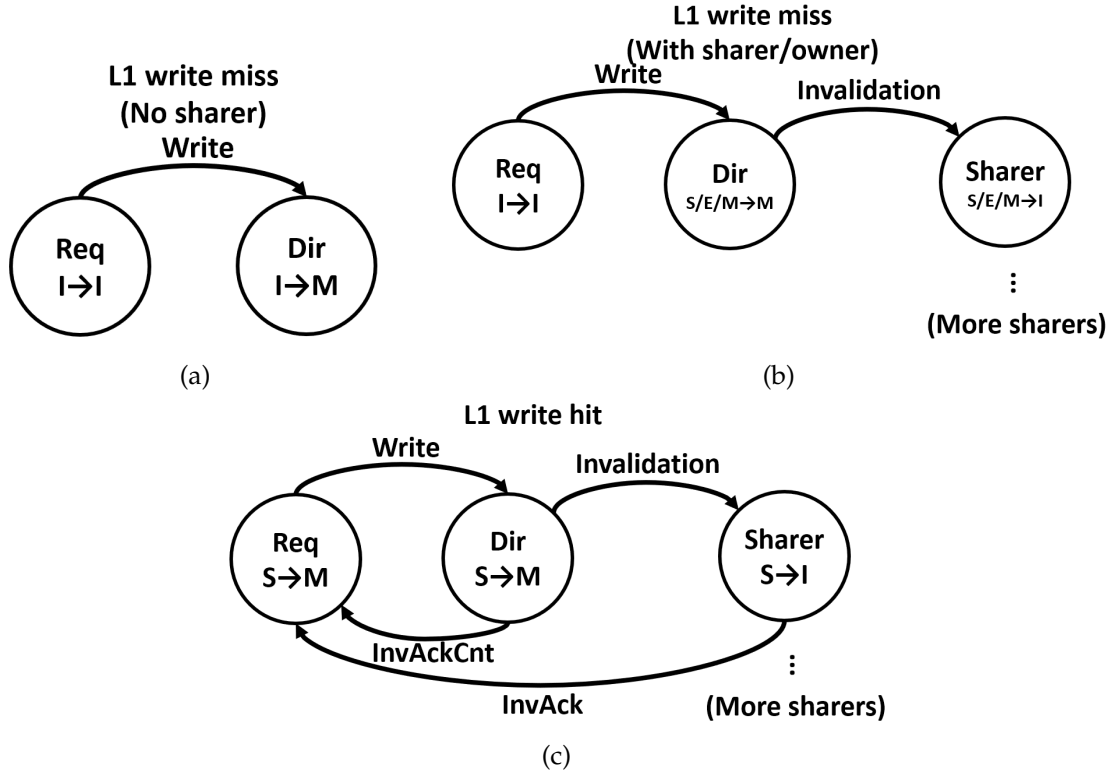


Figure 4.7.: Coherence State Transition when L1 Write with (a) Miss on I Cache Line, (b) on S/E/M Cache Line, and (c) Hit.

Write to E/M Cache Line

For a write to a line already held in E or M, the coherence handling is straightforward, as illustrated in Figure 4.8. Since the requester already has exclusive permission for the cache line, no invalidation or ownership transfer is required. At the L1 level, a line in E transitions immediately to M when it is written, while a line already in M simply remains in M. At the directory level, the line is likewise recorded in M, as the requester continues to be the sole valid owner.

The data path still follows the write-through policy of the private L1. The updated data is sent downward to L2, but it may not arrive there immediately because the write passes through the write buffer and experiences variable downstream latency. As a result, even if the directory state at L2 has already been updated to M, the latest data value is not necessarily available in L2 at that exact moment. This is the reason why later read requests to an E/M cache line cannot trigger GET to the current owner and force a write-buffer flush.

4. Implementation

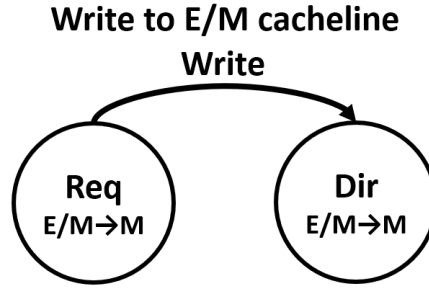


Figure 4.8.: Coherence State Transition when L1 Write to E/M Cache Line.

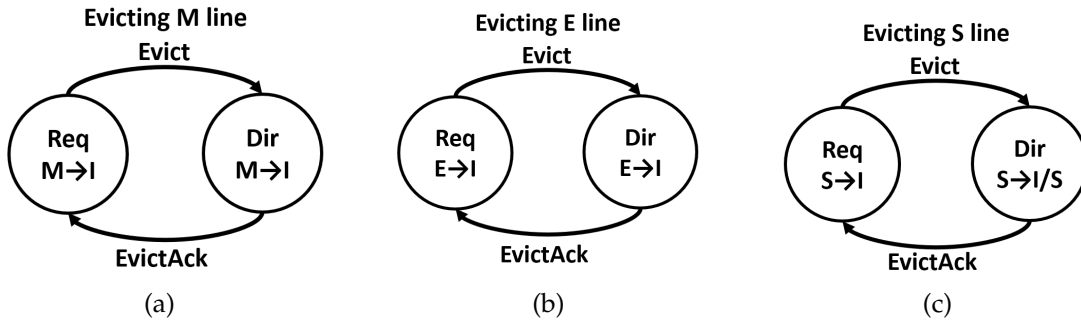


Figure 4.9.: Coherence State Transition when Evicting (a) M , (b) E, and (c) S State Cache Line.

Eviction and Replacement

For L1 eviction and replacement, the coherence behavior is largely the same regardless of whether the evicted line is in M, E, or S, as shown in Figure 4.9. In all three cases, the private L1 removes the line locally and informs the L2 directory so that the corresponding sharer and coherence state information can be updated. Eviction leaves the cache line invalid in both L1. For an evicted E or M line, the directory also invalidates it. For an evicted S line, the directory removes the requester from the sharer list while the line may remain in S state depending on whether other sharers still exist.

Due to the write-through property, clean and dirty evictions are handled in the same way because the latest data is expected to have already been propagated. The eviction mainly serves to inform the L2 that the evicting L1 no longer holds the line.

The special case is replacement, where eviction is triggered by fetching new data into an already occupied L1 entry. In this situation, the coherence protocol avoids inserting extra invalidation at the L1 side and instead lets the HPDcache controller handle the replacement without having to wait for EvictAck. Eviction still gets sent to the L2 directory controller to invalidate the stale copy in L2. In this way, replacement is treated as a coherence-visible eviction event, while the actual line substitution remains managed by the cache itself.

4. Implementation

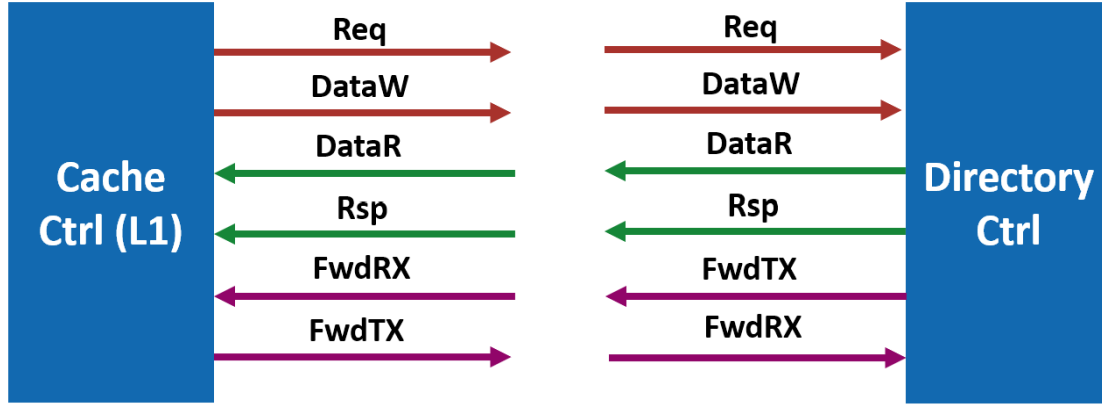


Figure 4.10.: Coherence message and data channels.

Transient States

In addition to the stable MESI states, there are transient states to represent cache lines that are in the middle of an ongoing coherence transaction. These states are necessary because many coherence operations leaves a cache line either waiting for acknowledgements or being targeted by a coherence action that has not yet fully completed. Without transient states, the controller would be unable to distinguish such in-flight situations from ordinary stable states. In the current design, the L1 cache uses six transient states to capture these intermediate situations in detail, while the L2 directory controller uses one transient state in addition to the stable ones.

Please refer to Table 4.2 for the detailed coherence state transition table for the L1 cache controller and Table 4.3 for that of the directory controller. Each entry of the table describes the action, and the *italicized* text at the bottom of each box denotes the next coherence state. The transient states are denoted as XY^Z where X represents the previous state, Y represents the next stable state, and Z indicates the condition that cache line is waiting for before completing the transition. An A in the superscript means the cacheline is waiting for acknowledgment(s), and a D means that it is waiting for data.

4.2.3. Message and Data Channels

The proposed coherence protocol uses a structured message organization in order to support coherence transactions without introducing deadlock or cyclic dependencies. At a high level, all coherence traffic is divided into three message groups: request, forward, and response. As illustrated in Figure 4.10, these groups are mapped onto a set of message and data channels between the private L1 cache controller and the L2 directory controller. Although all messages travel over the same physical network, they are treated as logically separate traffic classes at the receiver side. This separation is important, because coherence operations often involve requests, forwarded probes, acknowledgements, and data responses that depend on one another in different ways.

The three message classes are shown in Figure 4.10 with different colors:

4. Implementation

State	Read Send read to Dir IS^D	Write Send Write to Dir I	Get	Evict	Replace	Inv	Evict Ack	Exc Data	Data	Inv Ack	Last Inv Ack
I		Send Write to Dir I	-	-	-	-	-	-	-	-	-
IS^D	Stall	Stall	Send GetAck to Dir	Stall	-	Send InvAck to Req $E \rightarrow S$	-	E	S	-	-
S	Hit	Send Write to Dir SM^A	-	Send Evict to Dir SI^A	Send Evict to Dir	Send InvAck to Req I	-	-	-	-	-
SM^A	Hit	Stall	-	Stall	-	Send InvAck to Req I	-	-	-	AckCnt-	M
M	Hit or Refill Req	Hit	Send GetAck to Dir Flush Wbuf S	Send Evict to Dir MI^A	Send Evict to Dir	Send InvAck to Req I	-	-	-	-	-
E	Hit	Hit M	Send GetAck to Dir S	Send Evict to Dir EI^A	Send Evict to Dir	Send InvAck to Req I	-	-	-	-	-
MI^A	Stall	Stall	Send GetAck to Dir Flush Wbuf	Stall	-	-	I	-	-	-	-
EI^A	Stall	Stall	Send GetAck to Dir	Stall	-	-	I	-	-	-	-
SI^A	Stall	Stall	-	Stall	-	Send InvAck to Req II^A	I	-	-	-	-
II^A	Stall	Stall	-	Stall	-	-	I	-	-	-	-

Table 4.2.: Detailed Coherence Protocol Table (L1 Cache Controller).

4. Implementation

State	Read	Write	GetAck	Evict S	Evict M (Owner)	Evict M (Non-owner)	Evict E (Owner)	Evict E (Non-owner)
I	Send exclusive data. E	Update L2 data. M	-	-	-	Send EvictAck	-	Send EvictAck
S	Send data Update sharers.	Update L2 data. Send Inv to sharers. M	-	Remove Req from sharers. Send EvictAck.	-	Remove Req from sharers. Send EvictAck.	-	Remove Req from sharers. Send EvictAck.
E	Send Get to L1 ES^A	Update L2 data. Send Inv to owner. Update sharers. M	-	Send EvictAck.	Write to Memory. Send EvictAck. I	Send EvictAck.	Send EvictAck. Clear sharers (owner).	Send EvictAck.
ES^A	Stall	Stall	Send latest data. Update sharers. S	-	Send EvictAck. Respond to pending read Req with latest data. E (new owner)	Send EvictAck. Respond to pending read Req with latest data. E (new owner)	Send EvictAck. Respond to pending read Req with latest data. E (new owner)	Send EvictAck. Respond to pending read Req with latest data. E (new owner)
M	Send Get to L1 ES^A	Update L2 data. Send Inv to owner. Update sharer.	-	Send EvictAck.	Write to Memory. Send EvictAck. I	Send EvictAck.	-	Send EvictAck.

Table 4.3.: Detailed Coherence Protocol Table (L2 Directory Controller).

4. Implementation

Req (Evict)	Coherence Rsp	FwdRX (\$)	FwdTX (\$)	FwdRX (D)	FwdTX (D)
Addr	Addr	Addr	Addr	Addr	Addr
CoreID	CoreID	Type	Type	Type	Type
Valid	TID	Line State	Line State	Line State	Line State
	IsInvAckCnt	New Owner	New Owner	New Owner	New Owner
	InvAckCnt	Core ID	Core ID	Core ID	Receivers
		Num InvAck	Num InvAck	Num InvAck	

Table 4.4.: Coherence message flits.

1. **Request (Class 1):** the red arrows. Messages that initiate a coherence transaction, including Read, Write, and Evict.
2. **Forward (Class 2):** the purple arrows. Coherence actions sent to a non-requesting cache or messages between different L1 caches, including Invalidation, InvAck, Get, and GetAck.
3. **Response (Class 3):** the green arrows. Messages that terminate or complete a transaction, such as Data and EvictAck.

The design rule is that Class 2 never waits for Class 1, and Class 3 acts as a sink that should never wait for any of the other Classes. This strict ordering discipline helps avoid potential cyclic dependencies and the resulting deadlocks.

Table 4.4 shows the message flit structures for different coherence messages. For Read request, Write request, and Data, the existing message channels of the cache have been used to avoid unnecessary complexity and resource usage.

4.3. Cache Coherence Implementation

This section presents the implementation aspects of the proposed coherence protocol. The focus of this section shifts to the technical aspects and the main engineering choices required to realize the design in CachePool. In particular, the implementation must integrate the directory-based protocol into the existing cache hierarchy, coordinate coherence state updates with normal cache access, and provide a practical communication structure for coherence messages across the tile.

The discussion is organized as follows. The first subsection introduces the directory controller, focusing on its interaction with the L2 cache controller. The second subsection describes directory bank access, that is, how the cache controller and the directory controller share and coordinate access to the L2 metabank where the directory information is embedded. The third subsection presents the coherence message network, which is responsible for routing coherence requests, forwarded messages, and coherence responses between the directory and the private L1 caches.

4. Implementation

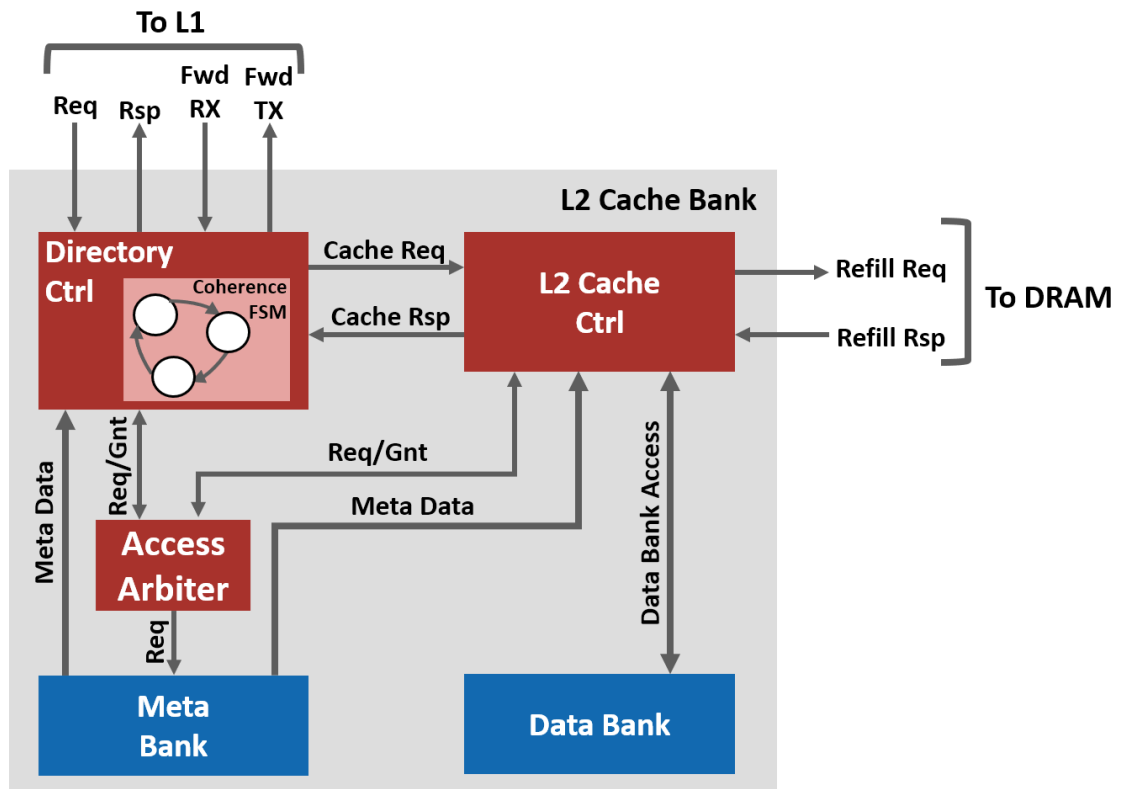


Figure 4.11.: L2 cache bank with directory controller.

4. Implementation

4.3.1. Directory Controller

After the introduction of coherence support, each L2 cache bank is equipped with a directory controller that is placed on the upstream side of the cache controller, as shown in Figure 4.11. The main design principle is that all transactions targeting the L2 bank must first pass through this controller, so that cache accesses become fully visible to the coherence logic. In this organization, the directory controller sits between the upstream traffic sources and the original L2 cache controller. Incoming requests are first interpreted by the directory controller, which then either handles them directly at directory level or forwards the corresponding cache request to the L2 cache controller when normal cache access is required.

On the upstream side, incoming cache and coherence messages from the interconnect are connected to the directory controller. Toward the cache bank, it issues internal cache requests and receives the corresponding cache responses. In this way, the original L2 cache controller remains responsible for the conventional cache functionality, while the newly inserted directory controller adds coherence awareness on top of it. This gives a clean separation of roles: the directory controller manages coherence interpretation and control flow, whereas the cache controller continues to manage cache-line refills, data-bank accesses, and DRAM-side refill traffic.

A key aspect of this organization is that the directory controller and the cache controller share the same meta bank. The meta bank now stores not only the conventional cache metadata, such as tags, but also the coherence metadata embedded into each L2 line as discussed in Section 4.2.1. Because both controllers may need to access this shared metadata, an additional access arbiter is inserted between them and the meta bank. This arbiter (explained in Section 4.3.2) manages potentially simultaneous accesses and ensures that meta-bank operations are serialized correctly. The data bank remains under the sole control of the L2 cache controller, since directory operations do not manipulate the cache-line data directly.

Internal Structure of Directory Controller

The core logic inside the directory controller is a coherence state machine (the small box inside the Directory Controller in Figure 4.11) that interprets incoming messages and determines the required coherence action for the targeted cache line. Whenever a valid message arrives on one of the directory input channels, the corresponding ready-valid handshake is accepted if the directory controller is not busy. In the same cycle, the controller issues a read to the meta bank to retrieve the coherence metadata, including its current coherence state and sharer information.

Once the metadata is returned, the coherence FSM decodes the operation (see Table 4.3) based on the request type and current line state within the same cycle. Based on this decision, the controller may trigger one or more actions:

- Request data from the L2 cache (Shared or Exclusive);
- Send invalidation to current sharers;

4. Implementation

- Invalidate/downgrade current owner;
- Probe (GET) the owner for latest line state;
- Send eviction acknowledgement;
- Send expected number of invalidation acknowledgements to the sharer;
- Write data to L2 cache;
- Update coherence state and/or sharer list.

If the transaction changes the coherence state or sharer vector of the cache line, the new metadata is written back into the meta bank. In this way, the FSM acts as the central decision point that converts the current directory state and the received coherence event into a sequence of cache-side and network-side operations.

Cache responses from the L2 cache controller do not pass through the coherence FSM. They bypass the coherence logic and are wired directly to the upstream response ports, keeping the data return path simple.

L1 Coherence Management

At the L1 side, the coherence support is integrated directly into the existing HPDcache control path rather than implemented as a separate external block. The coherence FSM is embedded into the cache controller module, `hpdcache_ctrl`, so that coherence handling becomes part of the normal L1 cache-control flow. This allows local cache accesses, incoming coherence events, and cache-line state transitions to be resolved within a unified controller.

To support this integration, corresponding changes are also made to the HPDcache memory controller, `hpdcache_memctrl`, in order to coordinate access to the meta bank. In this way, the modified L1 controller preserves the original HPDcache organization while extending it with the coherence functionality required by the CachePool private-cache hierarchy.

4.3.2. Directory Bank Access

For both L1 and L2 cache, the coherence metadata shares the same memory structure as the cache metadata, meaning that the cache controller and the directory controller must access the same storage structure. To handle this, the design uses a round-robin arbitration scheme between the normal cache-side access path and the coherence-side access path. At any given time, only one of the two controllers is granted access to the meta bank, while the other request is held until it can be served.

The arbiter returns a `gnt` signal to indicate that a request has been accepted and is being served. This signal serves as a control flag for the requesting controller to load the correct metadata once the access completes. Requests that are not granted immediately

4. Implementation

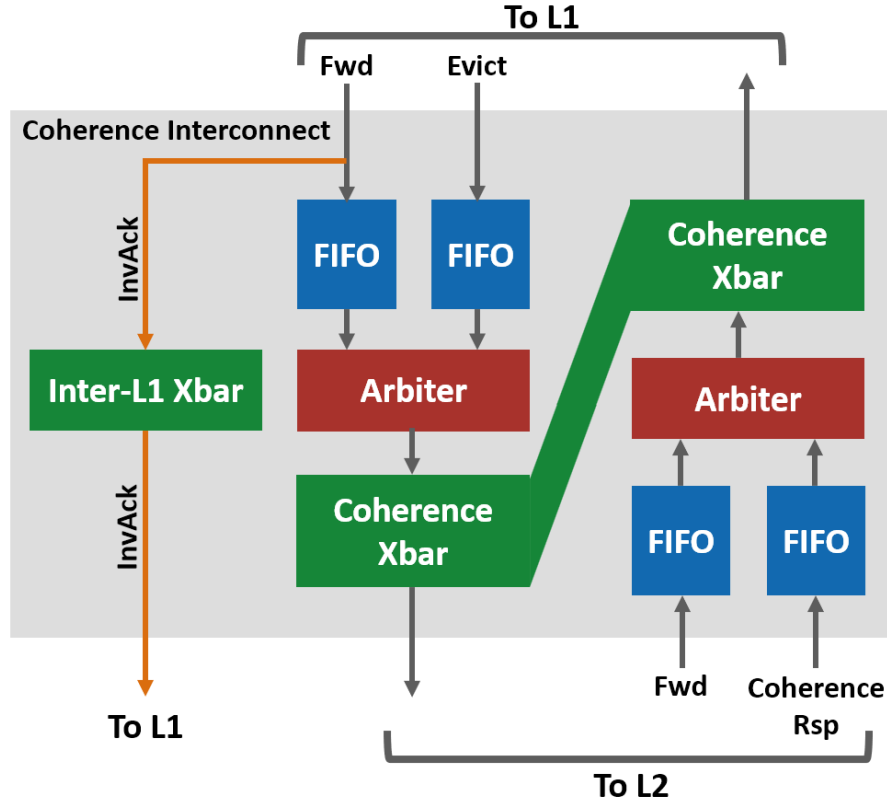


Figure 4.12.: Interconnect network for coherence messages.

are latched and held valid until they can be issued later, ensuring that no access is lost when simultaneous demand arises from both cache and coherence logic.

For metadata write-back, masking or byte-write is used to prevent contamination between fields. This is important because cache metadata and coherence metadata share the same meta-bank entry. A coherence update may need to modify only the coherence state or sharer bits, while a cache-side update may need to modify tag or replacement-related fields. Masked write-back allows only the intended portions of the entry to be updated, without corrupting unrelated metadata stored in the same line.

To ensure that the correct victim way is selected on coherence write-backs, the implementation reuses the same victim-selection logic of the cache controllers.

4.3.3. Coherence Message Network

The coherence protocol uses a dedicated coherence interconnect to route coherence traffic independently from the cache interconnect. The structure of the interconnect is shown in Figure 4.12. This separation keeps coherence communication decoupled from ordinary cache and refill traffic. In the current design, this interconnect carries three main classes of coherence traffic as mentioned in Section 4.2.3: **forward messages**, **eviction requests**,

4. Implementation

and coherence responses.

As shown by Figure 4.12, incoming messages are first buffered using FIFOs to tolerate concurrent requests. This is necessary because messages of different classes may be generated simultaneously by different sources. After buffering, round-robin arbiters select between competing messages and forward the chosen message into the coherence crossbar. In this way, the interconnect can absorb short bursts of coherence traffic while still providing a simple arbitration mechanism for onward routing.

For messages exchanged between L1 and L2, the design reuses the same crossbar structure as the cache interconnect [2]. As a result, the same bank interleaving mechanism is preserved, ensuring that each coherence message is routed to the correct L2 bank according to its address. The address bits that are displaced during scrambling are carried inside the TCDM metadata and later reconstructed for upward coherence operations, such as invalidations.

A special case arises for inter-L1 coherence communication as emphasized by the orange arrows in Figure 4.12. In the current protocol, InvAck messages are exchanged between private L1 caches. A separate inter-L1 crossbar dedicated to this traffic is therefore required.

Invalidation Interconnect

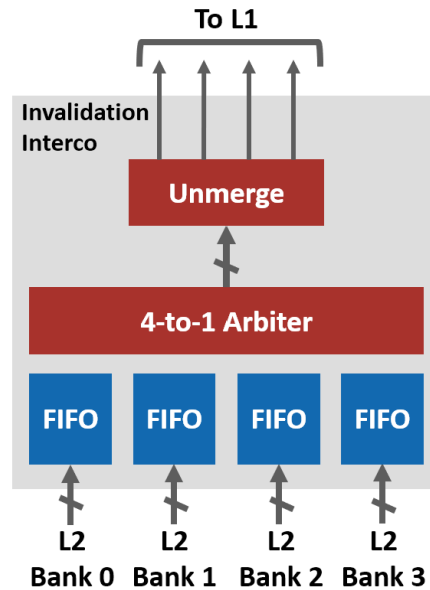


Figure 4.13.: Interconnect network for Invalidation messages.

A special case in the coherence network is the routing of invalidation messages, which requires its own dedicated interconnect. Unlike ordinary coherence messages, invalidations are not naturally 1-to-1. Each L2 directory controller may need to invalidate several private L1 caches for the same coherence transaction, and multiple L2 banks may

4. Implementation

generate such invalidations concurrently. Under the current tile configuration, this leads to a worst-case **16-to-4** communication pattern: four L2 banks, each potentially issuing up to four invalidations, targeting the four private L1 caches in the tile.

For this reason, invalidation traffic is handled by the separate invalidation interconnect shown in Figure 4.13. At the input side, each L2 bank feeds its outgoing invalidation package into a dedicated FIFO, so that concurrent invalidation bursts from different directory controllers can be absorbed without immediate loss or blocking. These buffered packages are then presented to a 4-to-1 arbiter, which selects one L2-bank package at a time for transmission through the network.

The selected package is first passed through an unmerge stage, which expands the chosen package into up to four individual invalidation messages, one for each target private L1 cache. These messages are then routed to the corresponding L1 caches.

Results

This chapter presents the evaluation of the proposed cache hierarchy design. The goal is to assess the impact of the introduced private L1 cache and coherence mechanism from both the architectural and implementation perspectives. To this end, the chapter considers the functional behavior of the design, its performance, and hardware costs.

The discussion is organized into three parts. It first describes the evaluation setup, including the simulation or implementation environment, the selected workloads, and the main measurement methodology used in this work. It then presents the performance results, where the effects of the proposed design on execution behavior, memory access efficiency, and overall system performance are analyzed. The corresponding results are discussed in Section 5.2. Finally, the chapter reports the area, timing, and power characteristics of the implementation, providing an evaluation on the hardware overhead and physical design implications of the proposed architecture. These results are presented in Section 5.3.

5.1. Evaluation Setup

This section presents the evaluation setup used in this work, including the hardware configuration of the CachePool system and HPDcache, and the test workloads used to evaluate both the function and the performance of the design. Together, these form the basis for assessing the architectural behavior and implementation quality of the proposed cache hierarchy and coherence mechanism.

5.1.1. Hardware Configurations

The evaluated system is configured as a single-tile CachePool cluster with four core complexes (CCs) in total. Both the data width and the address width of the cores are 32-bit. The evaluated platform corresponds to the smallest complete coherent CachePool instance, which is sufficient to expose the full intra-tile behavior of the proposed private-L1 coherence design.

5. Results

At the tile level, the original shared CachePool data cache remains 256 KiB, 4-way associative, with a 512-bit cache-line width and four banks per core. The lower level is connected to the main memory through with the refill interconnect width being 128-bit. Please refer to Appendix A for the full hardware configuration parameters.

Each private L1 (implemented using HPDcache) is configured as a 64-set, 4-way associative cache with 4 words per cache line and a word width of 128 bit, resulting in a cache-line size of 512 bit (64 B) and a total capacity of 16 KiB per L1. The memory-side interface width is therefore also 512-bit, matching the width of a cache line. The cache is operated in write-through mode. The number of requesters is set to 2, corresponding to the scalar Snitch path and the coalesced Spatz path. Please refer to Appendix B for a full list of HPDcache parameters.

5.1.2. Testbenches and Software Workloads

The evaluation workload consists of two parts: functional verification and software-based performance evaluation. The former is used to validate the functional correctness of the implemented cache hierarchy and coherence protocol, while the latter is used to assess the architectural impact of the design under representative workloads.

Functional verification is carried out using a set of coherence-oriented test cases designed to exercise the state transitions described in Section 4.2.2, as well as a number of corner cases. The tested scenarios include:

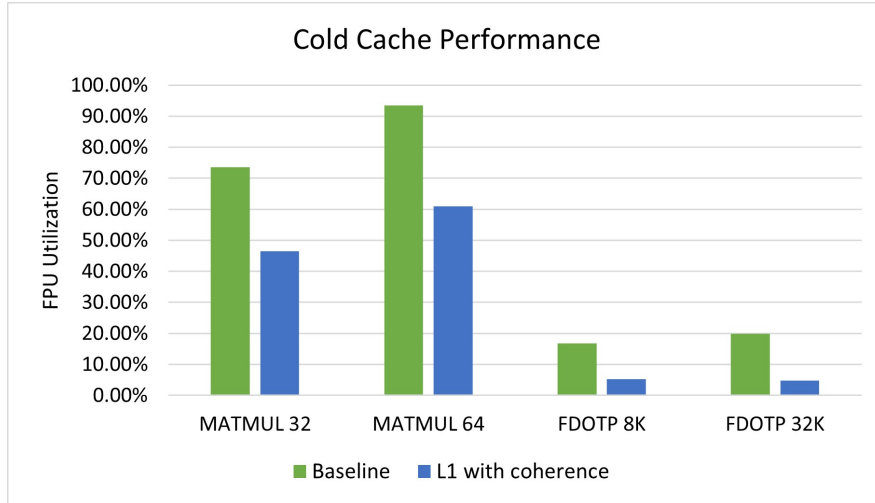
- Continuous read and write operations;
- Write collisions to the same address and the same cache set;
- Read collisions to different words within the same cache line;
- Read-Write conflicts to the same address and the same cache set.

These cases are intended to cover both ordinary coherence behavior and contention-heavy situations that stress the protocol logic. In addition to unit cases, software kernels such as Floating-point dot product (F-DOTP), Floating-point matrix multiplication (F-MATMUL), General matrix-vector multiplication (GEMV), and Fast Fourier transform (FFT) are also used for functional verification.

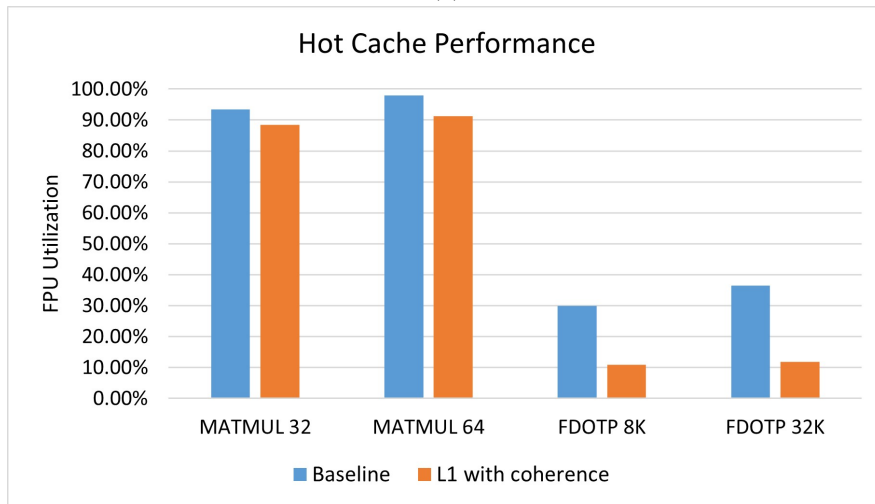
5.2. Performance

The performance impact of the proposed private L1 cache and the extended cache hierarchy is evaluated using representative compute kernels with different data sizes and access characteristics. In particular, F-DOTP is tested with vector sizes of 8K and 32K to capture streaming and reuse behavior under increasing working-set sizes, while F-MATMUL is evaluated with $MNK = 32$ and 64 to reflect more compute-intensive workloads with higher data locality. These benchmarks are selected to highlight how the private L1 caches affect the overall execution performance and behavior.

5. Results



(a)



(b)

Figure 5.1.: Performance for selected kernels under (a) cold cache and (b) hot cache.

5. Results

Figure 5.1 shows FPU utilization under cold-cache and hot-cache conditions for both the baseline design and the modified design with private L1 and coherence. A performance degradation can be observed across all tested workloads.

In the cold-cache scenario shown in Figure 5.1a, the degradation is more pronounced. For MATMUL 32 and MATMUL 64, utilization drops from around 73.6% to 46.4% and from 93.4% to 61.0%, respectively. The impact is even more evident for FDOTP 8K and FDOTP 32K. This indicates that the added cache hierarchy introduces a considerable amount of memory access latency. On a cache miss, the requests must travel through the private L1 and the directory controller before reaching L2. During cold execution, when most accesses miss in L1, this extra indirection does not provide reuse benefits but instead increases access latency, reducing the FPU utilization.

For the hot-cache scenario shown in Figure 5.1b, overall utilization improves as expected due to increased cache hit rate. For MATMUL workloads, the degradation becomes smaller (from 97.9% to 91.3%), indicating that when data reuse is high, the private L1 can partially amortize its overhead. Nevertheless, the remaining gap suggests that coherence-related overheads, such as metadata lookup, transient state handling, and message passing, still introduce additional latency.

5.2.1. Performance Impact Breakdown

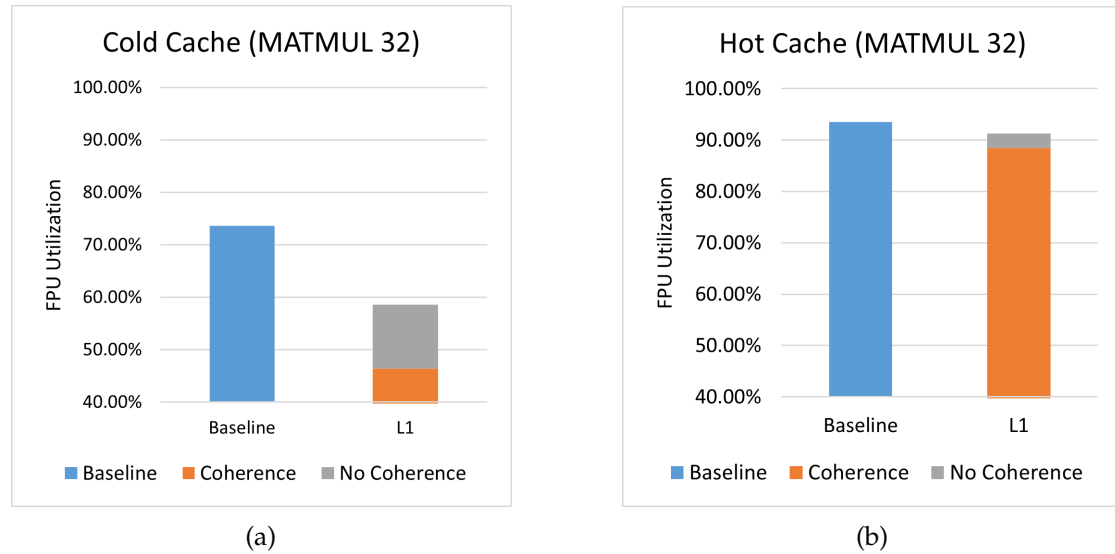


Figure 5.2.: Performance for matmul with and without coherence under (a) cold cache and (b) hot cache.

The impact of the proposed design can be further broken down by separating the effect of introducing the private L1 cache itself from the additional cost of coherence. Figure 5.2 shows this breakdown for MATMUL 32, comparing three configurations: the baseline design, L1 without coherence, and L1 with coherence. This separation helps

5. Results

identify that the observed performance degradation originates from two distinct sources: the overhead of the added L1 cache and the overhead of coherence management.

L1 Overhead

Comparing the baseline with the L1 without coherence configuration reveals the cost of inserting the private L1 cache into the memory hierarchy. In the cold-cache case, a significant drop in FPU utilization is already observed before coherence is enabled. This indicates that the additional cache level introduces non-negligible latency overhead. In the baseline design, requests are serviced directly by the shared cache, whereas with L1, all accesses must first pass through the private cache, even on misses. This adds extra cycles due to tag lookup, arbitration, and forwarding to L2.

Although the L1 is designed to maintain a one-cycle hit latency, its benefit is not visible in cold-cache execution where most accesses miss. Instead, the miss penalty dominates. In the hot-cache case, the gap between baseline and L1 without coherence becomes smaller, suggesting that when reuse is present, the L1 begins to amortize its overhead. However, even in this case, the utilization does not fully recover to baseline levels.

Coherence Overhead

The additional drop from L1 without coherence to L1 with coherence reveals the cost of the coherence mechanism itself. In both cold and hot scenarios, this difference is clearly visible, although smaller than the L1 overhead. The primary contributor here is the added latency and control complexity associated with coherence handling.

Coherence introduces several sources of overhead. Even in the best case, an access may require two cycles instead of one due to metadata lookup and coherence-state checking. Second, the shared meta bank becomes a point of contention between cache access and coherence updates, leading to possible congestion and stalled accesses. Third, coherence operations generate additional network traffic, such as invalidations, acknowledgements, and probes, which increase pressure on the interconnect and introduce stalls. Finally, the protocol employs relatively conservative control flow preventing aggressive overlap of memory operations.

5.3. Physical Design

This section presents the back-end evaluation results of CachePool after integrating the private L1 caches and the associated coherence functionality. The goal is to examine the backend cost introduced by these architectural changes from a physical-design perspective, including their impact on area, timing, and power. The modified CachePool is fully routed using GF12 technology using Synopsys Fusion Compiler under the clock frequency of 1 GHz.

The discussion focuses on the cost of introducing the private per-CC L1 caches, the coherence-related control logic, and the additional interconnect and metadata structures

5. Results

required by the new design. The detailed results and analysis are presented in the following subsections.

5.3.1. Area

The synthesized design occupies a total logic area of 11416kGE. Compared with the baseline CachePool tile without private L1 caches, this corresponds to an area increase of 2638kGE compared to the baseline. As expected, the added area is mainly associated with the data SRAM of the four private per-CC L1 caches. Figure 5.4 further breaks the total area down by component, showing that 16% of the area is contributed by the private L1 cache SRAM. All of the coherence support constitutes only 1.5% of the total area, showing the low area cost of the proposed design.

L1 Cache Controller

Figure 5.3a shows the post-physical-synthesis layout with the four private L1 caches highlighted. The data macros of the four L1 caches, corresponding to CC0–CC3, are labeled directly in the figure, while the synthesized L1 controller logic is colored according to the legend on the right. The control logic associated with each L1 appears as a relatively compact logic region, indicating that the added per-cache control remains localized.

From the layout, the physical cost of the private L1 cache is dominated by the data array macros, while the surrounding L1 controller logic occupies a comparatively smaller area. This is expected, since the main storage overhead comes from introducing four dedicated private caches, whereas the associated control logic mainly provides cache management, interfacing, and coherence support.

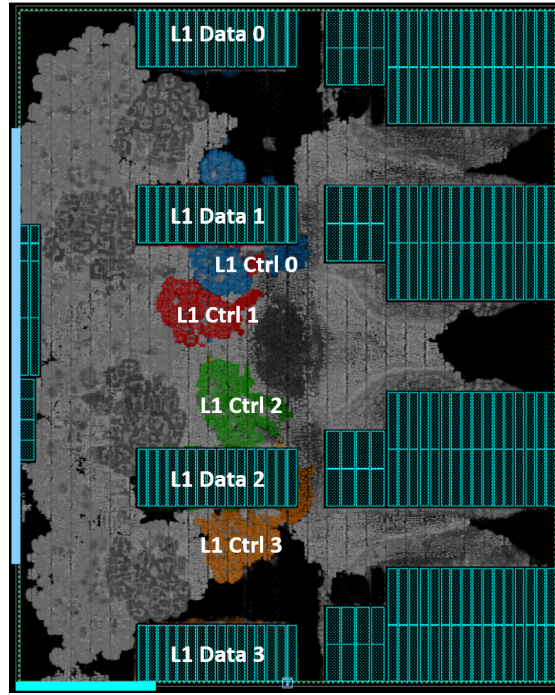
L2 Cache Controller and Directory Controller

Figure 5.3b shows the same layout with the cells related to the shared L2 cache highlighted for clarity. The L2 cache SRAM components are visible on the right side of the layout, each consisting of a narrow tag array placed next to a much larger data array.

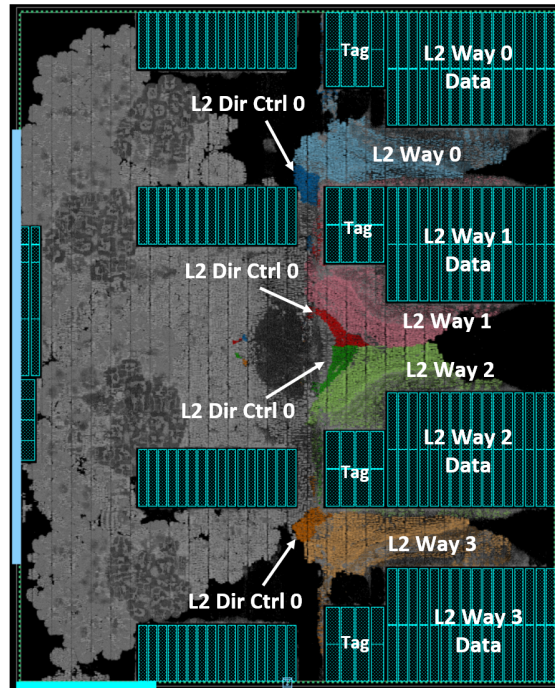
The colored regions around these memory macros correspond to the cells of the L2 cache controllers and the newly added directory controllers. As indicated by the legend, each bank is assigned a separate color pair. Each L2 bank is accompanied by its own local controller logic, with the directory controller placed close to the corresponding cache controller and tag/data macros. Physically, such proximity is beneficial because both the cache controller and the directory controller need frequent access to the shared tag/meta path, while the cache controller additionally drives the data-bank access and refill path.

The figure also gives an intuitive view of the relative cost of the added coherence support. Compared with the large area occupied by the L2 memory macros themselves, the directory-controller logic appears as a comparatively localized logic region surrounding each bank, occupying a modest region around each L2 bank, and remains

5. Results



(a)



(b)

Figure 5.3.: Layout of a CachePool tile with (a) L1 caches colored and (b) L2 caches colored.

5. Results

Baseline (kGE): total 8778

Core	L2 Ctrl	L2 Data	L2 Tag	Others
2.6K	1.1K	3.3K	620	1K
30%	13%	38%	7%	12%

With L1 and Coherence (kGE): total 11416

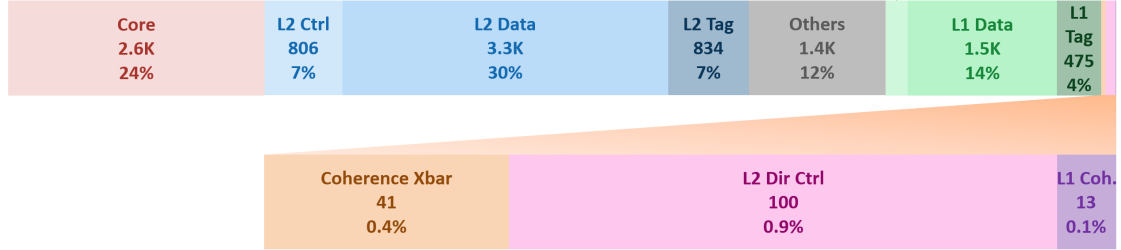


Figure 5.4.: Area breakdown of CachePool.

small compared with the larger footprint of the cache controllers. This suggests that embedding the coherence control at L2 level is physically lightweight.

5.3.2. Timing

The timing results indicate that the design successfully closes timing at the typical corner, while showing a moderate degradation at the worst-case operating condition (SS, 0.72, 125). At the worst corner, the design achieves a maximum frequency of 890 MHz, compared to 944 MHz for the baseline implementation. This corresponds to an approximate 50 MHz frequency reduction.

A closer inspection of the critical path reveals a negative slack of -0.119 ns. The critical path originates from the HPDcache MSHR logic and propagates through the core before terminating at the HPDcache data SRAM. Appendix C includes a more detailed summary of the critical path. This path highlights that the issue is related to the HPDcache timing path. One of the ways to resolve this timing violation would be to cut the path using a spill register.

Conclusion and Future Work

This work presented the design and implementation of a private-cache-based memory hierarchy for CachePool. A private write-through L1 data cache for each core complex was introduced and the original tile-level shared cache was repurposed as a logically shared, inclusive L2. On top of this hierarchy, a lightweight directory-based MESI-style coherence protocol was developed to maintain consistency across the private caches while preserving the shared-memory programming model required by the target workloads.

The proposed design was evaluated through both functional verification and software-kernel-based performance analysis. Coherence-oriented test cases were used to exercise the major protocol state transitions and corner cases, while representative kernels, including floating-point compute kernels were used to assess execution behavior under realistic traffic patterns. In addition, the back-end implications of the new design were studied through physical-design analysis in order to quantify the hardware cost of the private caches and coherence support. The performance evaluation demonstrates a performance degradation caused by both the private L1 cache and the coherence protocol. From the physical design perspective, the coherence functionality contributes to 1.5% of the total area, keeping the cost minimal.

In summary, this work demonstrates that extending CachePool with private L1 caches and lightweight directory-based coherence is feasible both architecturally and in implementation. It provides a foundation for future exploration of more advanced write-through policies, larger multi-tile systems, and further optimization of the cache-coherence and back-end cost trade-offs.

Several directions remain open for future work. A natural next step is to complete the exploration of lazy write-through policies and study how they interact with the current coherence design. Another important direction is to extend the coherence mechanism beyond the single-tile setup toward multi-tile scaling, where the hierarchical local/global sharer organization introduced in this work can be evaluated.

Cachepool Hardware Configuration

Listing A.1: CachePool hardware configurations used.

```
#####
## CachePool Cluster ##
#####

# Number of tiles
num_tiles ?= 1
# Number of cores
num_cores ?= 4
# Core datawidth
data_width ?= 32
# Core addrwidth
addr_width ?= 32

#####
## CachePool Tile ##
#####

# Number of cores per CachePool tile
num_cores_per_tile ?= 4
# Refill interconnection data width
refill_data_width ?= 128
##### L2 Data Cache #####
# L2 data cacheline width (in Bit)
l2d_cacheline_width ?= 512
# L2 data cache size (in KiB)
l2d_size ?= 256
# L2 data cache banking factor (how many banks per core?)
l2d_bank_factor ?= 1
# L2 coalescing window
l2d_coal_window ?= 2
# L2 data cache number of ways per
```

A. Cachepool Hardware Configuration

```
l2d_num_way ?= 4
# L2 data cache size per tile (KiB)
l2d_tile_size ?= 256
# L2 data cache tag width (TODO: should be calcualted)
l2d_tag_data_width ?= 64

#####
## CachePool CC ##
#####
# Spatz fpu support?
spatz_fpu_en ?= 1
# Spatz number of FPU
spatz_num_fpu ?= 4
# Spatz number of IPU
spatz_num_ipu ?= 4
# Spatz max outstanding transactions
spatz_max_trans ?= 32
# Snitch/FPU max outstanding transactions
snitch_max_trans ?= 16

#####
## Main Memory ##
#####
# number of channels
dram_channel ?= 4
# bank width (DRAM width, change with care)
dram_bank_width ?= 512
# interleaving factor (in order of bank_width)
dram_interleave ?= 16

#####
## Peripherals ##
#####
# Hardware stack size (in Byte)
stack_hw_size ?= 1024

# Stack size (total, including share and private, 32'h800)
stack_tot_size ?= 2048
```

Appendix B

HPDcache Configuration

Listing B.1: HPDcache hardware configurations used.

```
localparam hpdcache_pkg::hpdcache_user_cfg_t HPDcacheUserCfg = '{
    nRequesters: HPDCACHE_NREQUESTERS,
    paWidth: 32,
    wordWidth: L0WordWidth,
    sets: 64,
    ways: 4,
    clWords: 4,
    reqWords: 1,
    reqTransIdWidth: tidWidth + numWordOffsetBits,
    reqSrcIdWidth: 1,
    victimSel: hpdcache_pkg::HPDCACHE_VICTIM_PLRU,
    dataWaysPerRamWord: 1,
    dataSetsPerRam: 64,
    dataRamByteEnable: 1'b1,
    accessWords: 4,
    mshrSets: 8,
    mshrWays: 4,
    mshrWaysPerRamWord: 4,
    mshrSetsPerRam: 8,
    mshrRamByteEnable: 1'b1,
    mshrUseRegbank: 1'b1,
    cbufEntries: 2,
    refillCoreRspFeedthrough: 1'b1,
    refillFifoDepth: 2,
    wbufDirEntries: 4,
    wbufDataEntries: 4,
    wbufWords: 4,
    wbufTimecntWidth: 3,
    rtabEntries: 4,
    flushEntries: 2,
    flushFifoDepth: 2,
```

B. HPDcache Configuration

```
memAddrWidth: L1AddrWidth,  
memIdWidth: tidWidth,  
memDataWidth: L1LineWidth,  
wtEn: 1'b1,  
wbEn: 1'b0,  
lowLatency: 1'b1  
};
```

Critical Path

Startpoint: HPDcache MSHR (check_cache_set registers)
 Endpoint : HPDcache data SRAM clock gate

Corner: sspg_sigcmax_0p72v_125c
 Path Type: max

```

-----
Data arrival time : 1.433 ns
Data required time: 1.314 ns
-----
Slack (VIOLATED) : -0.119 ns
-----
  
```

```

Path Summary:
hpdcache_mshr.check_cache_set_q_reg
  (module: hpdcache_miss_handler / i_hpdcache_mshr)
  ↓
local MSHR decision / set-check combinational logic
  ↓
cachepool_cc / Spatz interface
  signal: fp_lsu_mem_rsp_i[9]
  ↓
Spatz FPU sequencer logic
  (module: i_cachepool_cc_i_spatz / gen_fpu_sequencer_i_fpu_sequencer)
  ↓
FPU memory request generation
  signal: fp_lsu_mem_req_o[0]
  ↓
cache control / ready-handshake logic
  
```

C. Critical Path

(between cachepool_cc and hpdcache control)
↓
hpdcache_ctrl internal control logic
(module: hpdcache_ctrl)
↓
hpdcache_memctrl SRAM control generation
signal: i_hpdcache_memctrl_i_data_cs_10_
↓
data SRAM request path
signal: gen_data_sram.../req_i[0]
↓
data SRAM clock-gate enable
endpoint: gen_data_sram.../i_ckg_i_clk_gate/E

Appendix	D
----------	----------

Task Description



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Institut für Integrierte Systeme
Integrated Systems Laboratory

TASK ASSIGNMENT FOR A
MASTER THESIS AT THE DEPARTMENT OF
INFORMATION TECHNOLOGY AND ELECTRICAL ENGINEERING

AUTUMN SEMESTER 2025

Ho Tin Hung

***CachePool: A scalable domain-specific cache-coherent
manycore architecture with scalar-vector capability
for dataplane***

September 1, 2025

Advisors: Zexin Fu, OAT U25, zexifu@iis.ee.ethz.ch
Diyoun Shen, ETZ J 78, dishen@iis.ee.ethz.ch

Professor: Prof. Dr. L. Benini

Handout: September, 2025

Due: March, 2026

The final report is to be submitted electronically. All copies remain property of the Integrated Systems Laboratory.

1 Introduction

Wireless baseband dataplane processing is a prime domain for energy-efficient parallel computing: the Radio Access Networks (RAN) market is vast, and the throughput and efficiency requirements are escalating. As 5G evolves toward 5.5G and 6G, per-cell computational complexity is projected to grow far beyond what technology scaling alone can deliver, demanding architectural and implementation advances across the stack. This project addresses that challenge with a focus on dataplane tasks in the Radio Link Control (RLC) layer. Service Data Units (SDUs) from higher layers are queued and assembled into Protocol Data Units (PDUs) with sequence headers for retransmission management before being passed to MAC, an interaction that tightly couples performance and reliability constraints.

Viewed as a computing workload, RLC exhibits three defining features. First, scale: modern deployments must sustain on the order of 10 million packets/s overall, with each RLC instance exceeding 10^3 packets/s and thousands of instances active concurrently. Second, latency: the Transition Time Interval (TTI) spans $62.5\ \mu\text{s}$ to 1 ms (typically $500\ \mu\text{s}$), setting a hard real-time envelope for all sub-tasks. Third, irregularity: per-user tasks are independent but highly variable, the code footprint is small, while data structures (e.g., linked lists) create sparse, non-contiguous accesses over large address ranges. Together, these properties imply hundreds of concurrent threads touching dispersed state with poor spatial locality.

For such access patterns, explicitly managed hierarchies (e.g., scratchpad memory + DMA) are ill-suited: tiling and double-buffering become inefficient as data dependencies branch unpredictably. A hardware-managed, cache-based hierarchy is required; however, enabling hundreds of Processing Elements (PEs) to access shared cache banks efficiently is challenging. To address this, we propose a private L1 cache with a lazy write-through policy, coupled with a shared inclusive L2 cache, and a lightweight hardware-managed cache coherence scheme.

2 Project Description

We propose and investigate an cache hierarchy tailored to RLC dataplane requirements: private L1 caches per core complex with a *lazy write-through* policy, backed by a logically shared, inclusive, multi-banked L2 and a lightweight hardware-managed coherence scheme. The goal is to sustain high throughput processing within the TTI envelope while preserving a simple shared-memory model for highly irregular, sparse accesses.

Architecture. Each tile hosts four Snitch+Spatz core complexes (CCs). Every CC owns a private L1 cache. All L1s connect to a *shared, inclusive* L2 that is logically single yet physically *multi-banked* (e.g., one bank per tile) to scale bandwidth. Inclusion guarantees

any valid L1 line resides in L2; we can therefore extend the L2 *tag RAM* with a few per-line directory bits instead of adding a separate coherence directory.

Lazy write-through policy. Conventional write-through can overload controllers and L2 bandwidth under write-intensive phases. We defer propagation using *time-based* and/or *write count-based* thresholds: stores accumulate in the L1 and, upon threshold, are merged at cache-line granularity and written to L2, and then the cache-line in the L1 can be kept valid and clean, or even self-invalidated to improve the coherence performance. Proper (possibly adaptive) thresholds reduce L1→L2 traffic, shorten effective core→L1 write latency, and improve energy efficiency. Because the design remains write-through (no long-lived dirty state), L2 observes the latest data early and primarily issues *invalidations* to other L1s. In predictable phases, proactive flush/invalidate can further reduce coherence tail latency.

Coherence and interconnect. The hardware coherence should be lightweight: L2 tracks sharers via *tag-embedded* bits and sends invalidations on L1 writes. Memory ordering relies on RISC-V atomics/fences. The specific topology (bank-local crossbar vs. minimal NoC), bank sizing, and placement will be explored under PPA constraints.

2.1 Milestones

The following are the milestones that we expect to achieve throughout the project:

- **M1 Literature Study:** Build a solid understanding of the *CachePool* architecture, as well as the building blocks that include snitch, spat, in-situ cache, and HPDcache. Explore the suitable L1 cache options.
- **M2 WT-L1 Architecture:** Develop the two-level cache architecture; L1 is private to each core, the L2 is shared by all the cores, and L1 is inclusive to L2 cache.
- **M3 Coherence Development:** Design, implement, and verify the lightweight coherence protocol.
- **M4 Lazy-WT:** Design and implement a basic lazy write-through policy.
- **M5 Benchmarking and PPA Analysis:** Performance evaluation based on vector and RLC kernel software, and PPA analysis on physical implementation.

Project phase	Time estimate
M1 Literature Study	3 week
M2 WT-L1 Architecture	4 week
M3 Coherence Development	5 week
M4 Lazy-WT	3 week
M5 Benchmarking and PPA Analysis	3 week
<i>Stretch goals</i>	<i>remaining time</i>
Write report	4 week
Prepare presentation	2 week

Table 1: Proposed time schedule and investment

2.2 Stretch Goals

Should the above milestones be reached earlier than expected and you are motivated to do further work, we propose the following stretch goals to aim for:

- **SG 1:** Explore a more adaptive and intelligent lazy write-through policy.
- **SG 2:** Scale up the single-tile design into a multi-tile cluster design, explore the hierarchical interconnect design.
- **SG 3:** Evaluate and explore the hardware and software cache coherence design under a multi-tile scale cluster.
- **SG 4:** Further design space exploration based on more real-world workloads.

3 Project Realization

3.1 Time Schedule

The time schedule presented in Table 1 is merely a proposition; it is primarily intended as a reference and an estimation of the time required for each required step.

3.2 Meetings

Weekly meetings will be held between the student and the assistants. The exact time and location of these meetings will be determined within the first week of the project in order to fit the student's and the assistants' schedule. These meetings will be used to evaluate the status and progress of the project. Besides these regular meetings, additional meetings can be organized to address urgent issues as well.

3.3 Weekly Reports

The student is advised, but not required, to write a weekly report at the end of each week and to send it to his advisors. The idea of the weekly report is to briefly summarize the work, progress and any findings made during the week, to plan the actions for the next week, and to bring up open questions and points. The weekly report is also an important means for the student to get a goal-oriented attitude to work.

3.4 Coding Guidelines

HDL Code Style Adapting a consistent code style is one of the most important steps in order to make your code easy to understand. If signals, processes, and modules are always named consistently, any inconsistency can be detected more easily. Moreover, if a design group shares the same naming and formatting conventions, all members immediately *feel at home* with each other's code. At IIS, we use lowRISC's style guide for SystemVerilog HDL: <https://github.com/lowRISC/style-guides/>.

Software Code Style We generally suggest that you use style guides or code formatters provided by the language's developers or community. For example, we recommend LLVM's or Google's code styles with `clang-format` for C/C++, PEP-8 and `pylint` for Python, and the official style guide with `rustfmt` for Rust.

Version Control Even in the context of a student project, keeping a precise history of changes is *essential* to a maintainable codebase. You may also need to collaborate with others, adopt their changes to existing code, or work on different versions of your code concurrently. For all of these purposes, we heavily use *Git* as a version control system at IIS. If you have no previous experience with Git, we *strongly* advise you to familiarize yourself with the basic Git workflow before you start your project.

3.5 Report

Documentation is an important and often overlooked aspect of engineering. A final report has to be completed within this project.

The common language of engineering is de facto English. Therefore, the final report of the work is preferred to be written in English.

Any form of word processing software is allowed for writing the reports, nevertheless the use of $\text{\LaTeX}$ with Inkscape or any other vector drawing software (for block diagrams) is strongly encouraged by the IIS staff.

If you write the report in $\text{\LaTeX}$, we offer an instructive, ready-to-use template, which can be forked from the Git repository at <https://iis-git.ee.ethz.ch/akurth/iisreport>.

Final Report The final report has to be presented at the end of the project and a digital copy needs to be handed in and remain property of the IIS. Note that this task description is part of your report and has to be attached to your final report.

3.6 Presentation

There will be a presentation (15 min presentation and 5 min Q&A) at the end of this project in order to present your results to a wider audience. The exact date will be determined towards the end of the work.

4 Deliverables

In order to complete the project successfully, the following deliverables have to be submitted at the end of the work:

- Final report (at due date)
- Presentation slides (before presentation starts)
- Source code and documentation for all developed software and hardware
- Testsuites (software) and testbenches (hardware)
- Synthesis and implementation scripts, results, and reports

Zurich, September 1, 2025

Prof. Dr. L. Benini

List of Acronyms

ALU	.Arithmetic Logic Unit
AMO	.Atomic Memory Operation
ASIC	.application-specific integrated circuit
CC	.Core Complex
CSR	.Control and Status Registers
DFT	.design for testability
DMA	.Direct Memory Access
DZ	.Microelectronics Design Center at ETH Zurich
F-DOTP	.Floating-point Dot Product
F-MATMUL	.Floating-point Matrix Multiplication
FFT	.Fast Fourier Transform
FPGA	.field-programmable gate array
FPU	.floating-point unit
FSM	.Finite-State Machine
GEMV	.General Matrix-Vector Multiplication
IC	.integrated circuit
IIS	.Integrated Systems Laboratory
ISA	.Instruction Set Architecture

List of Acronyms

LLC	.Last-Level Cache
LSU	.Load Store Unit
MAC	.Medium Access Control
MACU	.Multiply-Accumulate Unit
mMTC	.massive machine-type communications
MSHR	.Miss Status Holding Register
PDU	.Protocol Data Unit
PE	.Processing Element
RAN	.Radio Access Networks
RLC	.Radio Link Control
SDU	.Service Data Unit
SNR	.signal-to-noise ratio
SPM	.Scratchpad Memory
SRAM	.Static Random Access Memory
SSR	.Stream Semantic Registers
TCDM	.Tightly Coupled Data Memory
TTI	.Transition Time Interval
UMTC	.ultra-reliable low-latency communications
VRF	.Vector Register File
xMBB	.extreme mobile broadband

List of Figures

2.1. Architecture of a CachePool compute tile before integrating private L1 caches [2].	5
2.2. Architecture of a CachePool Core-Complex (CC) [2].	6
2.3. Architecture of Spatz vector engine[5].	8
2.4. HPDcache Core [9].	10
4.1. Architecture of a CachePool Tile with Private L1 Cache.	14
4.2. (a) Request and (b) Response Datapath of L1 cache.	16
4.3. (a) Coherence Meta Data Stored in Directory and (b) Extended L2 Tag Line.	20
4.4. Extended Tag Line of L1 cache.	21
4.5. Top-level state transition FSM for a cache line in (a) L1 cache and (b) L2 directory controller.	22
4.6. Coherence State Transition when L1 Read Miss on (a) S Cache Line (b) I Cache Line, and (c) E/M Cache Line.	23
4.7. Coherence State Transition when L1 Write with (a) Miss on I Cache Line, (b) on S/E/M Cache Line, and (c) Hit.	25
4.8. Coherence State Transition when L1 Write to E/M Cache Line.	26
4.9. Coherence State Transition when Evicting (a) M , (b) E, and (c) S State Cache Line.	26
4.10. Coherence message and data channels.	27
4.11. L2 cache bank with directory controller.	31
4.12. Interconnect network for coherence messages.	34
4.13. Interconnect network for Invalidation messages.	35
5.1. Performance for selected kernels under (a) cold cache and (b) hot cache.	39
5.2. Performance for matmul with and without coherence under (a) cold cache and (b) hot cache.	40
5.3. Layout of a CachePool tile with (a) L1 caches colored and (b) L2 caches colored.	43
5.4. Area breakdown of CachePool.	44

List of Tables

4.1. Key configuration parameters of the private L1 data cache.	15
4.2. Detailed Coherence Protocol Table (L1 Cache Controller).	28
4.3. Detailed Coherence Protocol Table (L2 Directory Controller).	29
4.4. Coherence message flits.	30

Bibliography

- [1] P. Marsch, I. Da Silva, Bulakci, M. Tesanovic, S. E. El Ayoubi, T. Rosowski, A. Kaloylos, and M. Boldi, “5g radio access network architecture: Design guidelines and key considerations,” *IEEE Communications Magazine*, vol. 54, no. 11, pp. 24–32, 2016.
- [2] D. Shen, Z. Fu, and L. Benini, “Cachepool: Manycore cluster of customizable, lightweight scalar-vector pes for irregular data-plane workloads,” Integrated Systems Laboratory, ETH Zurich, Tech. Rep., 2025.
- [3] C. Zhang, “Pulp-platform-insitu-cache.” [Online]. Available: <https://github.com/pulp-platform/Insitu-Cache>
- [4] F. Zaruba, F. Schuiki, T. Hoefler, and L. Benini, “Snitch: A tiny pseudo dual-issue processor for area and energy efficient execution of floating-point intensive workloads,” *IEEE TRANSACTIONS ON COMPUTERS*, vol. 70, no. 11, pp. 1845–1860, Nov. 2021.
- [5] M. Cavalcante, M. Perotti, S. Riedel, and L. Benini, “Spatz: Clustering compact risc-v-based vector units to maximize computing efficiency,” *arXiv*, Sep. 2023.
- [6] Y. Zhang, M. Bertuletti, S. Riedel, M. Cavalcante, A. Vanelli-Coralli, and L. Benini, “Terapool-sdr: An 1.89tops 1024 rv-cores 4mib shared-l1 cluster for next-generation open-source sware-defined radios,” in *GLSVLSI '24: Proceedings of the Great Lakes Symposium on VLSI 2024*, Jun. 2024, pp. 86–91. [Online]. Available: <https://doi.org/10.1145/3649476.3658735>
- [7] L. Steiner, M. Jung, F. S. Prado, K. Bykov, and N. Wehn, “Dramsys4.0: An open-source simulation framework for in-depth dram analyses,” *International Journal of Parallel Programming*, vol. 50, pp. 217–242, Mar. 2022. [Online]. Available: <https://doi.org/10.1007/s10766-022-00727-4>
- [8] C. Fuguet, “Hpdcache: Open-source high-performance l1 data cache for risc-v cores,” in *The 20th ACM International Conference on Computing Frontiers (CF '23)*, 2023, pp. 377–378.
- [9] —, *HPDcache User Guide*, Jan. 2025.

Bibliography

- [10] D. Million, N. Oliete-Escuín, and C. Fuguet, “Breaking the memory wall with a flexible open-source l1 data-cache,” in *2024 Design, Automation Test in Europe Conference Exhibition (DATE)*, 2024, pp. 1–2.
- [11] A. Ros and S. Kaxiras, “Complexity-effective multicore coherence,” in *Proceedings of the 21st International Conference on Parallel Architectures and Compilation Techniques*, ser. PACT ’12. New York, NY, USA: Association for Computing Machinery, 2012, p. 241–252. [Online]. Available: <https://doi.org/10.1145/2370816.2370853>
- [12] T. project. [Online]. Available: <https://largo.lip6.fr/trac/tsar/wiki/CacheCoherence>
- [13] H. Liu, C. Dévigne, L. Garcia, Q. Meunier, F. Wajsbürt, and A. Greiner, “Rwt: Suppressing write-through cost when coherence is not needed,” in *2015 IEEE Computer Society Annual Symposium on VLSI*, 2015, pp. 434–439.
- [14] S. Kaxiras and G. Keramidas, “Sarc coherence: Scaling directory cache coherence in performance and power,” *IEEE Micro*, vol. 30, no. 5, p. 54–65, Sep. 2010. [Online]. Available: <https://doi.org/10.1109/MM.2010.82>
- [15] ARM, “Amba axi protocol specification.” [Online]. Available: <https://developer.arm.com/documentation/ih0022/latest/>
- [16] D. Sorin, M. Hill, and D. Wood, *A Primer on Memory Consistency and Cache Coherence*, ser. SYNTHESIS LECTURES ON COMPUTER ARCHITECTURE. Morgan Claypool, 2011.